ChainForge Developer Guide

Version 1.0 · October 2026 · A fictional product guide for documentation practice

> **Note:** ChainForge is a fictional blockchain platform created for documentation practice. All endpoints, domains, keys, addresses and package names in this guide are invented and are not real services.

About This Guide

This guide teaches developers how to build applications on ChainForge, from first API call to production launch. It is task-oriented: each chapter explains a concept, shows how to do the related task, and points out the mistakes developers most often make. Where you need exhaustive, per-endpoint detail (every parameter, header and error code), the companion **ChainForge API Reference Guide** is the right place.

Conventions used in this guide

- `Monospace text` is code, a command, a file name or an API field.
- Code samples are shown in `curl`, JavaScript/TypeScript and Python. Replace placeholder values such as `cfk_test_YOUR_KEY` with your own.
- **Note** marks helpful context. **Warning** marks something that can lose funds, leak secrets or break production.
- Amounts of the native token are expressed in **FORGE** in prose and in **sparks** (the smallest unit, where 1 FORGE = 10^18 sparks) in API payloads.

---

1. Introduction

1.1 What is ChainForge

ChainForge is a general-purpose, programmable blockchain platform. It combines three things that developers normally have to assemble themselves:

1. **A Layer 1 blockchain network** with fast block times (about 2 seconds) and deterministic finality (a transaction is irreversible about 4 seconds after inclusion).
2. **An EVM-compatible execution environment**, so you can write smart contracts in Solidity and use familiar tooling and libraries.

3. **A managed developer platform**: a REST API, WebSocket event streams, webhooks, SDKs, a command-line tool called `forge`, a hosted testnet and a block explorer, so you can build without running your own nodes.

Typical applications built on ChainForge include token payments and remittances, tokenized assets, loyalty and rewards programs, NFT marketplaces, supply-chain provenance trackers, and on-chain registries.

What ChainForge gives you

Capability	What it means for you
Fast finality	Users see a confirmed, irreversible result within seconds, which makes point-of-sale and checkout flows practical.
Low, predictable fees	Fees are paid in gas, with a fee market that keeps typical transfers at a small fraction of a cent.
EVM compatibility	Existing Solidity contracts, Hardhat/Foundry projects and wallet libraries work with little or no change.
Managed infrastructure	The ChainForge API fronts a fleet of nodes, so you do not provision, sync or monitor your own.
Event delivery	Subscribe over WebSocket or receive signed webhooks when on-chain events occur.
Testnet and devnet	Free test tokens and a local network let you build safely before touching real value.

What ChainForge is not

ChainForge is not a custody service, an exchange or a wallet product. You (or your users) control private keys. ChainForge never has access to them, and cannot recover funds if a key is lost.

1.2 Key Concepts and Terminology

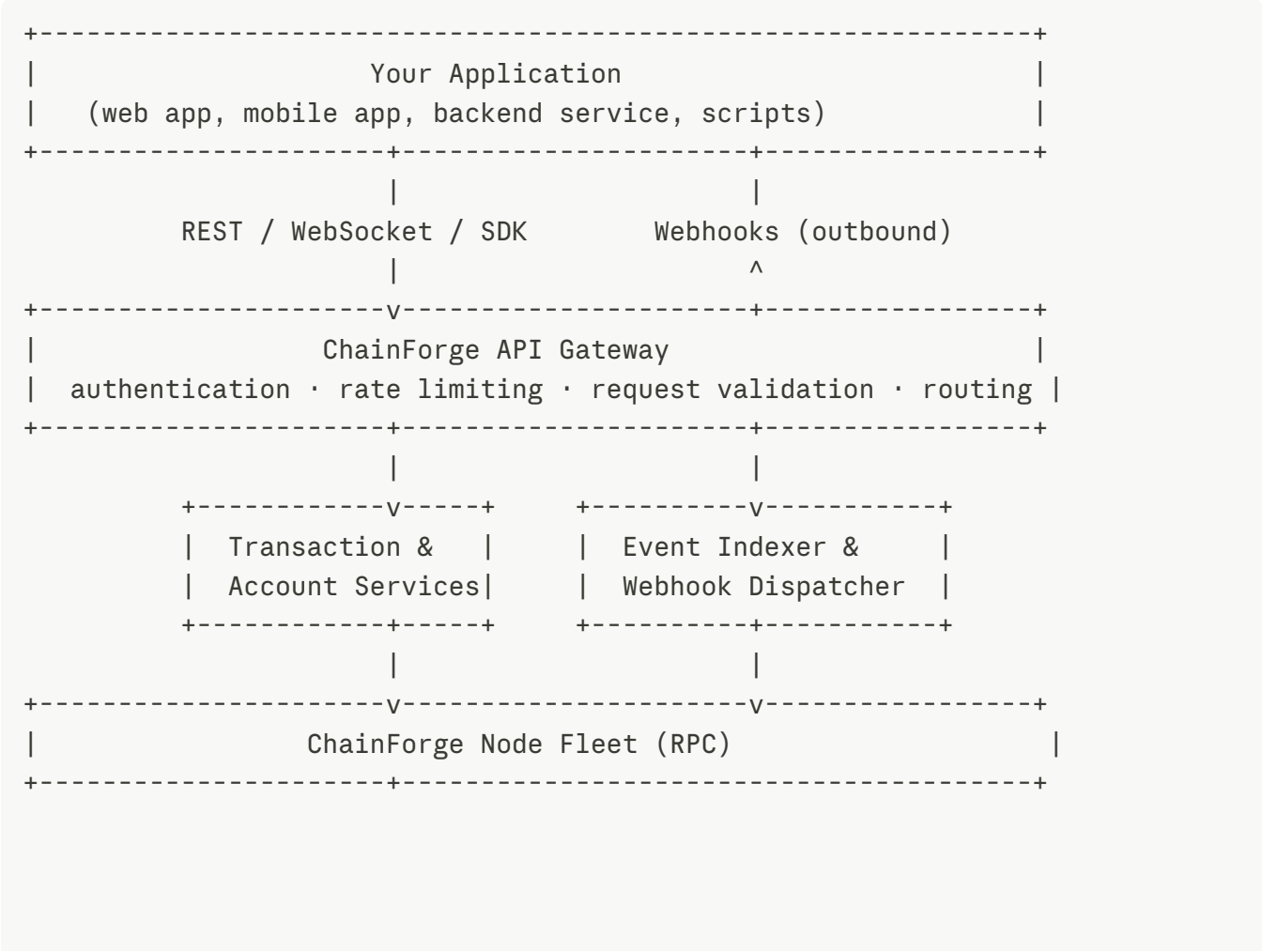
The following terms appear throughout this guide. Chapter 3 treats the most important ones in depth; this section gives you working definitions.

Term	Definition
Blockchain	A shared, append-only ledger maintained by many independent computers (nodes). Once data is finalized it cannot be altered.
Node	A computer running ChainForge software that stores the ledger and validates new data.
Validator	A node that has staked FORGE and takes part in producing and approving blocks.
Block	A batch of transactions, linked to the previous block by a cryptographic hash.
Transaction	A signed instruction that changes ledger state, such as transferring tokens or calling a contract.
Account	An identity on the chain, identified by an address. Accounts hold balances and (for contracts) code.
Address	A 42-character identifier beginning with `0x`, for example `0x71C7656EC7ab88b098defB751B7401B5f6d8976F`.
Private key	A secret number that proves ownership of an address and signs transactions. Never share it.
Wallet	Software or a service that stores private keys and signs transactions.
Smart contract	A program stored on the chain that runs when a transaction calls it.
Gas	A measure of computational work. Every operation costs gas.
Gas price	The amount, in sparks, you pay per unit of gas.
Nonce	A per-account counter that orders an account's transactions and prevents replays.
Finality	The point at which a transaction can no longer be reversed.

Term	Definition
Mempool	The waiting area where valid but not yet included transactions sit.
FORGE	The native token of ChainForge, used to pay fees and secure the network.
Spark	The smallest unit of FORGE. 1 FORGE = 1,000,000,000,000,000,000 sparks.
Testnet (Ember)	A public network with valueless test tokens, used for development.

1.3 Architecture Overview

Understanding the layers helps you decide where to integrate and where to look when something goes wrong.



```
                          |
+-------------------------v-----------------------------------------+
|        ChainForge Network (validators, consensus, ledger)         |
+-------------------------------------------------------------------+
```

The layers, top to bottom

1. **Your application** decides what to build and holds (or delegates) key management.
2. **The API gateway** authenticates every request using your API key or OAuth token, enforces rate limits, validates payloads and routes them.
3. **Platform services** handle common tasks: preparing and broadcasting transactions, reading account state, indexing contract events and dispatching webhooks.
4. **The node fleet** exposes the chain through standard JSON-RPC. If you prefer, you can call the RPC endpoint directly instead of the REST API.
5. **The network** is the set of validators that run consensus, order transactions into blocks and finalize them.

Two ways to talk to ChainForge

- **The ChainForge REST API** is the recommended path. It adds conveniences such as gas estimation, transaction tracking by ID, pagination, and consistent error formats.
- **The JSON-RPC endpoint** is fully Ethereum-compatible and is useful when you rely on libraries (such as ethers.js or web3.py) that expect RPC. This guide focuses on the REST API and SDKs.

How a transaction travels

1. Your app builds a transaction and signs it with a private key.
2. The API gateway accepts it and forwards it to the node fleet.
3. A node places it in the mempool and gossips it to validators.
4. A validator includes it in a block (about 2 seconds).
5. A second block confirms the first, and the transaction becomes final (about 4 seconds).
6. The indexer records the result, and any matching webhooks are delivered to your server.

1.4 Who This Guide Is For

This guide is written for:

- **Backend and full-stack developers** integrating payments, tokens or on-chain records into an application.
- **Smart contract developers** deploying and maintaining Solidity contracts on ChainForge.
- **DevOps and platform engineers** responsible for monitoring, key management and production readiness.
- **Technical architects** evaluating ChainForge for a project.

What you should already know

- Basic programming in JavaScript/TypeScript or Python.
- How REST APIs work (HTTP methods, JSON, status codes, headers).
- Helpful but not required: general blockchain ideas such as wallets and transactions. Chapter 3 covers the fundamentals.

How to use this guide

- New to blockchain? Read chapters 1 to 5 in order.
- Experienced with Ethereum? Skim chapter 3, then read chapters 2, 4, 6 and 8, which cover what is specific to ChainForge.
- Ready to launch? Go straight to chapter 12.

---

2. Getting Started

This chapter takes you from nothing to a confirmed transaction on the testnet in about 15 minutes.

2.1 Prerequisites

Before you begin, make sure you have:

Requirement	Details
A computer with internet access	macOS, Linux or Windows (WSL2 recommended on Windows)
Node.js 18 or later **or** Python 3.9 or later	Choose the SDK language you prefer
`curl`	Used for the examples that call the API directly

Requirement	Details
A code editor	Visual Studio Code is a good default
An email address	For your developer account
A terminal	You will run a few `forge` CLI commands

Check your versions:

```
node --version     # v18.0.0 or later
python3 --version  # 3.9 or later
curl --version
```

2.2 Create a Developer Account

1. Go to the ChainForge Developer Console at `https://console.chainforge.example`.
2. Select **Sign up**, enter your email address and choose a password, or continue with a single sign-on provider.
3. Open the verification email and confirm your address.
4. Enable **two-factor authentication** (strongly recommended) under **Account → Security**.
5. Create an **organization** (for example, "Acme Payments"). Organizations own API keys, webhooks and usage billing, and you can invite teammates to them later.
6. Create your first **project**, such as "my-first-app". Projects separate environments and keys, so each application should have its own.

> **Note:** New accounts start on the **Free tier**, which includes testnet access, 10 requests per second and community support. You can upgrade when you are ready for mainnet traffic (see chapter 12).

2.3 Generate API Keys

API keys identify your project to the platform.

1. In the Console, open your project and select **API Keys → Create key**.
2. Give the key a descriptive name, such as `dev-laptop` or `ci-pipeline`.

3. Choose the **environment**: `test` for the Ember testnet or `live` for mainnet. Start with `test`.
4. Choose **permissions**: for this guide, select **Read** and **Write**. Production keys should use the narrowest permissions that work.
5. Select **Create**, then **copy the key immediately**. It is shown only once.

Keys have a recognizable format:

Prefix	Meaning
`cfk_test_`	Testnet key. Safe for development.
`cfk_live_`	Mainnet key. Can move real value.

Store the key in an environment variable rather than in source code:

```
export CHAINFORGE_API_KEY="cfk_test_YOUR_KEY"
```

> **Warning:** Never commit API keys to a repository, paste them into chat tools or embed them in front-end code. If a key is exposed, revoke it immediately in the Console and create a new one.

Verify that your key works:

```
curl https://api.chainforge.example/v1/status \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

A successful response looks like this:

```
{
  "environment": "test",
  "network": "ember",
  "chainId": 7701,
  "latestBlock": 4820113,
  "apiVersion": "v1",
  "status": "operational"
}
```

If you receive `401 Unauthorized`, see section 11.1.

2.4 Install the SDK and CLI

Install an SDK

JavaScript / TypeScript

```
mkdir chainforge-quickstart && cd chainforge-quickstart
npm init -y
npm install @chainforge/sdk
```

Python

```
mkdir chainforge-quickstart && cd chainforge-quickstart
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install chainforge-sdk
```

Install the CLI

The `forge` CLI creates wallets, requests test tokens, deploys contracts and runs a local devnet.

```
npm install --global @chainforge/cli
forge --version
```

Configure it with your key:

```
forge auth login --api-key "$CHAINFORGE_API_KEY"
forge config set network ember
```

2.5 Connect to the Testnet

The **Ember** testnet is a public network that mirrors mainnet behavior but uses valueless test tokens (**tFORGE**).

Setting	Value
Network name	Ember (testnet)
Chain ID	7701

Setting	Value
REST base URL	`https://api.chainforge.example/v1`
JSON-RPC URL	`https://rpc.ember.chainforge.example`
WebSocket URL	`wss://ws.ember.chainforge.example`
Block explorer	`https://explorer.ember.chainforge.example`
Faucet	`https://faucet.ember.chainforge.example` or `forge faucet request`
Native token	tFORGE (no monetary value)

Create a wallet and fund it:

```
forge wallet create --name alice
# Address: 0x71C7656EC7ab88b098defB751B7401B5f6d8976F
# Keystore saved to ~/.forge/keystore/alice.json

forge faucet request --address 0x71C7656EC7ab88b098defB751B7401B5f6d8976F
# Requested 100 tFORGE. Expect it within 10 seconds.

forge balance 0x71C7656EC7ab88b098defB751B7401B5f6d8976F
# 100.0 tFORGE
```

> **Note:** The faucet dispenses 100 tFORGE per address every 24 hours. If you need more for load testing, use the local devnet described in chapter 9.

2.6 Quickstart: Send Your First Transaction

You will create a second wallet and send it 5 tFORGE from the first.

Using the CLI

```
forge wallet create --name bob
forge send --from alice --to 0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF --
amount 5
```

The CLI prints a transaction hash and waits for finality:

```
Transaction submitted: 0x9f3c...e21a
Waiting for confirmation... included in block 4820140
Finalized in block 4820141 (4.1s)
Status: success
```

Using JavaScript

```
import { ChainForge, Wallet, parseForge } from "@chainforge/sdk";

const client = new ChainForge({ apiKey: process.env.CHAINFORGE_API_KEY });

// Load the wallet you created with the CLI (it prompts for the keystore
password)
const alice = await Wallet.fromKeystore("alice",
process.env.KEYSTORE_PASSWORD);

const tx = await client.transactions.send({
  from: alice,
  to: "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  value: parseForge("5"), // converts FORGE to sparks
});

console.log("Submitted:", tx.hash);

const receipt = await tx.waitForFinality();
console.log("Status:", receipt.status, "Block:", receipt.blockNumber);
```

Using Python

```
import os
from chainforge import ChainForge, Wallet, parse_forge

client = ChainForge(api_key=os.environ["CHAINFORGE_API_KEY"])

alice = Wallet.from_keystore("alice", os.environ["KEYSTORE_PASSWORD"])

tx = client.transactions.send(
    sender=alice,
    to="0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
    value=parse_forge("5"),
)
print("Submitted:", tx.hash)

receipt = tx.wait_for_finality()
print("Status:", receipt.status, "Block:", receipt.block_number)
```

Using the REST API directly

When you use the REST API, you sign the transaction yourself and submit the raw signed payload. The SDKs do this step for you.

```
curl -X POST https://api.chainforge.example/v1/transactions \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: 6f1d2c9a-3b7e-4d55-9a21-0e8c4b7a1f90" \
  -d '{
    "signedTransaction": "0xf86c0a85..."
  }'
```

Response:

```
{
  "id": "txn_01JC7X3M9Q",
  "hash": "0x9f3c5d8a1b...e21a",
  "status": "pending",
  "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  "value": "5000000000000000000",
  "submittedAt": "2026-10-18T09:30:12Z"
}
```

Confirm the result

1. Open the block explorer at `https://explorer.ember.chainforge.example/tx/<your hash>` and look for **Status: Success** and **Finalized**.
2. Check Bob's balance: `forge balance <bob address>` should show `5.0 tFORGE`.
3. Check Alice's balance: it should be slightly under 95 tFORGE, because she paid a small gas fee.

What just happened

You built, signed and broadcast a transaction, watched it move from `pending` to `included` to `finalized`, and verified the balance change. Every other operation in this guide (calling contracts, minting tokens, reacting to events) follows the same pattern with different payloads.

Next steps: read chapter 3 to understand what you just did, or jump to chapter 5 for fuller coverage of accounts and transactions.

---

3. Core Concepts

This chapter explains the ideas behind every ChainForge operation. Reading it once will make the remaining chapters much easier to follow, and it will help you diagnose problems when something behaves unexpectedly.

3.1 Networks (Mainnet, Testnet, Local Devnet)

ChainForge runs several independent networks. Each has its own ledger, its own token supply and its own **chain ID**. A transaction signed for one network is invalid on another, which prevents accidental cross-network replays.

Network	Chain ID	Purpose	Token	Typical use
Forge Mainnet	7700	Production network with real economic value	FORGE	Live applications
Ember Testnet	7701	Public test network mirroring mainnet	tFORGE (no value)	Integration testing, demos, staging
Local Devnet	7702	Single-node network on your machine	dFORGE (no value)	Unit tests, rapid iteration, load tests

How to choose

- Develop and run automated tests against the **local devnet**. It is instant, free and resettable.
- Validate end-to-end behavior against **Ember** before every release. Ember has real network latency, real finality and real validators, so it catches problems devnet cannot.
- Use **Mainnet** only when you are ready to move real value, and only with `cfk_live_` keys.

Environment separation in the platform

Your API key's prefix selects the network. A `cfk_test_` key can only reach Ember, and a `cfk_live_` key can only reach Mainnet. This is deliberate: it makes it very hard to send a test request to production by mistake. Keep the two keys in separate secret stores and give them different access policies.

> **Warning:** Addresses are identical across networks. The same address on Ember and Mainnet refers to the same private key but to separate balances. Never assume that a successful testnet transfer proves your mainnet configuration is correct. Always verify the chain ID in your configuration.

3.2 Accounts and Wallets

Account types

ChainForge has two kinds of accounts, both identified by a 20-byte address written as 42 hexadecimal characters:

Type	Controlled by	Has code?	Typical role
Externally owned account (EOA)	A private key	No	A user's or service's wallet that signs and pays for transactions
Contract account	Its own code	Yes	A deployed smart contract

Only an EOA can start a transaction. A contract account acts only when a transaction (or another contract) calls it.

Keys and addresses

An EOA begins with a randomly generated 256-bit **private key**. A **public key** is mathematically derived from it, and the **address** is the last 20 bytes of a hash of the public key.

```
private key  --(one-way math)-->  public key  --(hash, last 20 bytes)-->
address
```

The derivation is one-way. Anyone can compute the address from the key, but nobody can recover the key from the address. This is why a published address is safe to share and a private key must never be shared.

Wallets

A **wallet** is anything that stores keys and signs transactions. Common types:

Wallet type	Description	Good for	Risk
Keystore file	Encrypted JSON file protected by a password (created by `forge wallet create`)	Development, scripts	Weak passwords, file leaks
Hardware wallet	Keys stay on a dedicated device	Treasury, high-value accounts	Physical loss without backup

Wallet type	Description	Good for	Risk
Custodial/KMS signer	Keys held in a cloud key-management service that signs on request	Backend services at scale	Misconfigured permissions
Browser/mobile wallet	Application that the end user controls	Consumer applications	Phishing, user error

Recovery phrases. Many wallets generate a 12 or 24-word recovery phrase that can recreate all keys. Whoever holds the phrase controls the funds. Treat it like a private key.

Nonces

Every account keeps a **nonce**, a counter that starts at 0 and increases by one for each transaction the account sends. The nonce does two jobs:

1. It enforces ordering. Transaction nonce 7 cannot execute before nonce 6.
2. It prevents replay. A signed transaction with nonce 7 can be included only once.

If you send transactions in parallel from one account, you must manage nonces yourself (section 5.3).

3.3 Transactions and Their Lifecycle

A **transaction** is a signed message that asks the network to change the ledger. Anatomy of a transaction:

Field	Meaning
`chainId`	Which network the transaction is valid on
`nonce`	The sender's transaction counter
`to`	Recipient address, or empty when deploying a contract
`value`	Amount of FORGE to transfer, in sparks
`data`	Encoded contract call or contract creation code (empty for plain transfers)
`gasLimit`	Maximum gas the sender is willing to consume

Field	Meaning
`maxFeePerGas`	Maximum price per gas unit the sender will pay
`maxPriorityFeePerGas`	Tip offered to the validator to prioritize the transaction
`signature`	Proof that the sender's key authorized all of the above

Transaction types

Type	`to`	`data`	Example
Value transfer	Recipient address	Empty	Send 5 FORGE to Bob
Contract call	Contract address	Encoded function call	Call `transfer()` on a token contract
Contract deployment	Empty	Contract bytecode	Deploy a new token contract

Lifecycle states

The ChainForge API reports one of these statuses on every transaction:

```
created --> pending --> included --> finalized
                |           |
                |           +--> failed   (executed, but reverted)
                +--> dropped / replaced   (never included)
```

Status	What it means	What you should do
`pending`	Accepted by the network, waiting in the mempool	Wait; poll or use webhooks
`included`	Placed in a block, but the block is not yet final	Treat as likely, not guaranteed
`finalized`	The block is final and cannot be reversed	Safe to fulfil orders

Status	What it means	What you should do
`failed`	Included and finalized, but the execution reverted. Gas is still consumed.	Read `revertReason`, fix the call and resend
`dropped`	Removed from the mempool without being included (for example, fee too low or expired)	Resend with a higher fee
`replaced`	Superseded by another transaction with the same nonce	Track the replacement

> **Note:** `failed` is different from `dropped`. A failed transaction **is on the chain**, and you paid for it. A dropped transaction left no trace and cost nothing.

3.4 Blocks, Confirmations and Finality

Transactions are grouped into **blocks**. Each block records its number (height), a timestamp, the hash of the previous block, the list of transactions, and a state root that summarizes the ledger after the block runs. Because each block includes the hash of the one before it, changing any past block would change every later hash, which is what makes history tamper-evident.

Block production

ChainForge uses a **proof-of-stake Byzantine fault tolerant (BFT)** consensus design:

- Validators stake FORGE to participate.
- A validator is selected as the proposer for each **slot** (about 2 seconds).
- Other validators vote on the proposed block.
- When at least two-thirds of the staked weight votes for a block, it is approved.

Confirmations and finality

Term	Meaning on ChainForge
Inclusion	Your transaction appears in a proposed block (about 2 seconds)

Term	Meaning on ChainForge
Confirmation	Additional blocks have been built on top of the including block
Finality	Two-thirds of the staked weight has voted twice on the block, so reverting it would require destroying a large fraction of all staked value. Reached about 4 seconds after submission.

ChainForge offers **deterministic finality**. Unlike chains where you wait for "six confirmations" and still accept a small probability of reversal, a finalized ChainForge transaction is final. This is why the API's `finalized` status is the one to rely on for anything with economic consequences, such as shipping goods or crediting an account.

Choosing when to act

Scenario	Safe to act on
Show "payment received" in a user interface	`included` (with a note that it is not final)
Ship a physical product	`finalized`
Credit a user's balance in your database	`finalized`
Display a leaderboard or non-financial update	`included`

3.5 Gas and Fees

Every operation a transaction performs costs **gas**, a unit of computational work. Fees prevent abuse (nobody can run infinite loops for free) and compensate validators.

The fee formula

```
fee (sparks) = gasUsed x effectiveGasPrice

effectiveGasPrice = min(maxFeePerGas, baseFee + maxPriorityFeePerGas)
```

- **`gasUsed`** is the gas the transaction actually consumed. A simple transfer uses 21,000. Contract calls use more, depending on their logic.

- **`baseFee`** is set automatically by the protocol. It rises when blocks are full and falls when they are not. The base fee is **burned** (removed from supply).
- **`maxPriorityFeePerGas`** is a tip that goes to the validator. A higher tip gets you included sooner when the network is busy.
- **`maxFeePerGas`** is your ceiling. You never pay more than this, and any unused amount is refunded.
- **`gasLimit`** is the maximum gas you allow. If execution needs more, the transaction fails and the gas up to the limit is consumed.

A worked example

A plain transfer on a quiet network:

Parameter	Value
`gasUsed`	21,000
`baseFee`	2 gwei-equivalent (2,000,000,000 sparks)
`maxPriorityFeePerGas`	1,000,000,000 sparks
`maxFeePerGas`	5,000,000,000 sparks

```
effectiveGasPrice = min(5,000,000,000, 2,000,000,000 + 1,000,000,000) =
3,000,000,000 sparks
fee = 21,000 x 3,000,000,000 = 63,000,000,000,000 sparks = 0.000063 FORGE
```

Estimating fees

You should never hard-code fees. Ask the API:

```
curl https://api.chainforge.example/v1/gas/estimate \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" \
  -H "Content-Type: application/json" \
  -d
'{"from":"0x71C7656EC7ab88b098defB751B7401B5f6d8976F","to":"0x2B5AD5c4795c026
514f8317c7a215E218DcCD6cF","value":"500000000000000000"}'
```

```
{
  "gasLimit": "21000",
  "baseFee": "2000000000",
  "suggested": {
    "slow":   { "maxFeePerGas": "3000000000", "maxPriorityFeePerGas":
"500000000" },
    "normal": { "maxFeePerGas": "5000000000", "maxPriorityFeePerGas":
"1000000000" },
    "fast":   { "maxFeePerGas": "8000000000", "maxPriorityFeePerGas":
"2500000000" }
  }
}
```

> **Note:** Always add a safety margin of about 10 to 20 percent to the `gasLimit` for contract calls, because the state of the chain can change between estimation and execution.

3.6 Smart Contracts

A **smart contract** is a program stored on the chain at its own address. It has **storage** (persistent state), **functions** (code anyone can call, subject to its own rules) and **events** (logs it emits for outside observers).

Properties you can rely on

- **Deterministic.** Given the same state and input, every validator computes the same result.
- **Transparent.** Contract code and state are publicly readable.
- **Immutable by default.** Deployed code cannot be changed. Upgradeability must be designed in (section 6.6).
- **Atomic.** A transaction either fully succeeds or fully reverts. There are no partial states.

Properties you must plan for

- **Costly storage.** Writing data on-chain is the most expensive operation. Keep large data off-chain and store only hashes or references.
- **No outbound network access.** A contract cannot call a web API. Use events and off-chain services instead.

- **Public, permanent mistakes.** A bug can be exploited by anyone, and cannot be quietly patched.

Read calls and write calls

Kind	Changes state?	Costs gas?	Needs a signature?	Speed
Read (`view` or `pure` function)	No	No	No	Immediate
Write	Yes	Yes	Yes	Needs inclusion and finality

Chapter 6 walks through writing, deploying and calling contracts step by step.

---

4. Authentication and Security

This chapter covers how to prove who you are to the ChainForge API, how to protect your keys and how to sign transactions safely. Security mistakes on a blockchain are often irreversible, so treat this chapter as required reading before you go to production.

4.1 API Keys and OAuth 2.0

ChainForge supports two authentication methods for the API. Both send a bearer token in the `Authorization` header.

Method 1: API keys

API keys are long-lived credentials created in the Console (section 2.3). They suit server-side applications and scripts that you control.

```
curl
https://api.chainforge.example/v1/accounts/0x71C7656EC7ab88b098defB751B7401B5
f6d8976F \
  -H "Authorization: Bearer cfk_test_YOUR_KEY"
```

Attribute	Detail
Format	`cfk_test_...` or `cfk_live_...`
Lifetime	Until revoked or rotated
Scope	Set when you create the key (`read`, `write`, `admin`)
Best for	Backend services, CI pipelines, scripts
Not suitable for	Browsers, mobile apps or any client the user controls

Method 2: OAuth 2.0

OAuth 2.0 issues short-lived access tokens with fine-grained scopes. Use it when third parties act on a user's behalf, or when your security policy prohibits long-lived secrets.

ChainForge supports two flows:

Flow	Use when
Client credentials	A backend service authenticates as itself (machine to machine)
Authorization code with PKCE	A user signs in to your app and grants it access to their data

Client credentials example

```
curl -X POST https://auth.chainforge.example/oauth/token \
  -d grant_type=client_credentials \
  -d client_id=YOUR_CLIENT_ID \
  -d client_secret=YOUR_CLIENT_SECRET \
  -d scope="transactions:read transactions:write"
```

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "transactions:read transactions:write"
}
```

Use the token exactly like an API key:

```
curl https://api.chainforge.example/v1/transactions/txn_01JC7X3M9Q \
  -H "Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Tokens expire after one hour. Request a new one before expiry, and cache it in memory in the meantime so you do not request a token per call.

Scopes

Scope	Allows
`accounts:read`	Read balances and account data
`transactions:read`	Read transactions and receipts
`transactions:write`	Submit transactions
`contracts:read`	Call read-only contract functions and read ABIs
`contracts:write`	Deploy and register contracts
`webhooks:manage`	Create, update and delete webhook endpoints
`admin`	Manage keys and project settings

Request only the scopes you need. A reporting dashboard needs read scopes alone.

4.2 Key Management Best Practices

There are two kinds of secrets to protect, and they are very different:

Secret	If leaked	Recoverable?
API key	Attackers can use your quota and read your data	Yes. Revoke and replace it.
Wallet private key	Attackers can move all funds controlled by that key	**No.** Transfers are irreversible.

Protecting API keys

1. **Keep them out of source control.** Use environment variables, a secrets manager or your CI platform's secret store. Add `.env` to `.gitignore`.
2. **Use separate keys per environment and per service.** If one leaks, you revoke one.
3. **Grant least privilege.** A service that only reads balances should not hold a `write` key.
4. **Restrict by IP address** (Console → API Keys → Allowed IPs) where your servers have fixed addresses.
5. **Rotate regularly.** Create the new key, deploy it, then revoke the old one. Rotate immediately after any suspected exposure or staff departure.
6. **Monitor usage.** The Console's **Usage** tab shows requests by key. An unexpected spike is a warning sign.

Protecting wallet private keys

1. **Never put a private key in code, logs, tickets or chat messages.**
2. **Do not keep large balances in hot wallets.** A hot wallet is one that a running server can sign with. Keep only the operating float there and hold the rest in cold storage or a hardware wallet.
3. **Prefer a KMS or hardware security module (HSM)** for server-side signing so the key never exists in your application's memory.
4. **Require multi-signature approval** for high-value operations. A multi-signature contract wallet needs several keys to authorize a transfer, so one stolen key is not enough.
5. **Back up recovery material offline**, in more than one physical location, and test your recovery process before you need it.
6. **Separate duties.** The person who can deploy contracts should not be the only person who can move treasury funds.

Comparing signing approaches

Approach	Security	Convenience	Recommended for
Keystore file in the app's file system	Low to medium	High	Local development only
Environment-variable private key	Low	High	Throwaway testnet scripts only
Cloud KMS / HSM signing	High	Medium	Production backends
Hardware wallet	Very high	Low	Treasury and administrative actions
Multi-signature contract wallet	Very high	Low to medium	Organizational funds

> **Warning:** If a private key has ever been pasted into a public repository, assume it is compromised even if you deleted the commit. Automated scanners find leaked keys within minutes. Move the funds to a new address immediately.

4.3 Signing Transactions

To authorize a transaction, your key creates a **digital signature** over the transaction's fields. Validators verify the signature against the sender's address. Because the signature covers every field, nobody can alter the amount, recipient or fee without invalidating it.

Where signing happens

Model	Description	Notes
Client-side signing	Your application signs locally, then submits the signed bytes	Keys never leave your environment. This is the model the SDKs use.
KMS signing	Your application asks a key service to sign the transaction hash	Keys never leave the HSM.

Model	Description	Notes
Wallet-prompt signing	A user's wallet shows the transaction and the user approves it	Standard for consumer apps.

The ChainForge API never receives or stores private keys. It receives only the already-signed transaction.

Signing with the SDK

```
import { ChainForge, Wallet, parseForge } from "@chainforge/sdk";

const client = new ChainForge({ apiKey: process.env.CHAINFORGE_API_KEY });
const wallet = new Wallet(process.env.WALLET_PRIVATE_KEY); // testnet only!

// 1. Build an unsigned transaction with correct nonce and fee suggestions
const unsigned = await client.transactions.prepare({
  from: wallet.address,
  to: "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  value: parseForge("1.5"),
});

// 2. Sign locally
const signed = await wallet.sign(unsigned);

// 3. Submit
const tx = await client.transactions.submitSigned(signed);
console.log(tx.hash);
```

```
from chainforge import ChainForge, Wallet, parse_forge
import os

client = ChainForge(api_key=os.environ["CHAINFORGE_API_KEY"])
wallet = Wallet(os.environ["WALLET_PRIVATE_KEY"])  # testnet only!

unsigned = client.transactions.prepare(
    sender=wallet.address,
    to="0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
    value=parse_forge("1.5"),
)
signed = wallet.sign(unsigned)
tx = client.transactions.submit_signed(signed)
print(tx.hash)
```

Signing messages (without a transaction)

You can prove you control an address without paying gas by signing a message. This is commonly used for sign-in with a wallet.

```
const message = "Sign in to Acme Payments\nNonce: 8c41f2\nExpires: 2026-10-
18T10:00:00Z";
const signature = await wallet.signMessage(message);

// Server side: recover the signer and compare
const signer = verifyMessage(message, signature);
if (signer.toLowerCase() === claimedAddress.toLowerCase()) { /* authenticated
*/ }
```

Always include a **server-issued nonce** and an **expiry** in the message so a captured signature cannot be replayed later.

Pre-signing checklist

Before signing any transaction, confirm:

- The **chain ID** matches the intended network.
- The **recipient address** is correct. Compare the full address, not only the first and last characters.
- The **amount** is in the right unit (sparks versus FORGE).
- The **contract call data** decodes to the function you expect.

- The **fee ceiling** is reasonable.

4.4 Rate Limits and Quotas

The API gateway limits request rates to keep the platform reliable for everyone.

Plan	Requests per second	Requests per day	Concurrent WebSocket subscriptions
Free	10	100,000	5
Growth	50	5,000,000	50
Enterprise	Custom	Custom	Custom

Every response includes headers that show where you stand:

Header	Meaning
`X-RateLimit-Limit`	Requests allowed per second on your plan
`X-RateLimit-Remaining`	Requests remaining in the current window
`X-RateLimit-Reset`	Seconds until the window resets
`Retry-After`	Present on `429` responses: seconds to wait before retrying

When you exceed the limit the API returns `429 Too Many Requests`:

```
{
  "error": {
    "code": "rate_limited",
    "message": "Rate limit of 10 requests per second exceeded.",
    "requestId": "req_7H2K9PQ"
  }
}
```

Handling limits gracefully

1. **Back off exponentially.** Wait the `Retry-After` period, then retry. If you are rate limited again, double the delay (1 s, 2 s, 4 s, 8 s) and add a small random jitter.

2. **Reduce calls.** Cache values that change slowly (chain ID, ABIs), and use batch endpoints where available.
3. **Subscribe instead of polling.** Webhooks and WebSocket subscriptions (chapter 8) replace repeated status checks.
4. **Spread load.** Do not fire hundreds of requests in the same instant. Use a queue with a concurrency limit.

```
async function withRetry(fn, maxAttempts = 5) {
  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      return await fn();
    } catch (err) {
      if (err.status !== 429 || attempt === maxAttempts) throw err;
      const base = (err.retryAfter ?? 2 ** (attempt - 1)) * 1000;
      const jitter = Math.random() * 250;
      await new Promise((r) => setTimeout(r, base + jitter));
    }
  }
}
```

The SDKs include automatic retry with backoff for `429` and `5xx` responses. You can tune it with the `maxRetries` option.

4.5 Security Checklist

Use this checklist before every release and revisit it quarterly.

Credentials

- [ ] API keys are stored in a secrets manager, not in code or images.
- [ ] Test and live keys are separate and stored separately.
- [ ] Each service has its own key with minimum scopes.
- [ ] IP allow-lists are configured for production keys.
- [ ] A key-rotation schedule exists and has been rehearsed.

Wallets and signing

- [ ] No production private key exists in source control, logs or developer laptops.
- [ ] Production signing uses a KMS, HSM or hardware wallet.
- [ ] Hot wallets hold only the operating balance.

- [ ] High-value transfers need multi-signature approval.
- [ ] Recovery material is backed up offline and the restore process is tested.

Application

- [ ] Inputs (addresses, amounts) are validated before signing.
- [ ] Amounts use integer arithmetic on sparks, never floating-point numbers.
- [ ] Webhook signatures are verified (section 8.3).
- [ ] Requests use idempotency keys (section 5.3).
- [ ] Error messages shown to users do not expose internal details or keys.

Smart contracts

- [ ] Contracts have been unit tested and tested on Ember.
- [ ] A third-party audit has been completed for contracts holding significant value.
- [ ] Admin functions are protected by multi-signature or time-locks.
- [ ] An incident response plan covers pausing contracts and rotating keys.

Operations

- [ ] Alerts exist for unusual spend, failed transactions and API error spikes.
- [ ] Logs exclude secrets and are retained for the period your compliance rules require.
- [ ] Team members use two-factor authentication on the Console.

---

5. Working with Accounts and Transactions

This chapter turns the concepts of chapter 3 into working code. You will create and manage wallets, read balances, send and track transactions, estimate gas and handle the awkward cases: stuck, failed and replaced transactions.

5.1 Create and Manage Wallets

Create a wallet with the CLI

```
forge wallet create --name treasury-test
forge wallet list
forge wallet show treasury-test
forge wallet export treasury-test --format keystore --out ./treasury-
test.json
```

The CLI stores an encrypted keystore in `~/.forge/keystore/`. Choose a strong password. A keystore is only as safe as its password.

Create a wallet in code

```
import { Wallet } from "@chainforge/sdk";

const wallet = Wallet.createRandom();
console.log("Address:", wallet.address);

// Encrypt before writing anywhere
const keystoreJson = await wallet.encrypt(process.env.KEYSTORE_PASSWORD);
```

```
from chainforge import Wallet
import os

wallet = Wallet.create_random()
print("Address:", wallet.address)

keystore_json = wallet.encrypt(os.environ["KEYSTORE_PASSWORD"])
```

> **Warning:** `Wallet.createRandom()` returns an object that contains the private key. Do not log the object, serialize it into responses or send it to error-tracking services.

Register an account with the platform (optional)

Registering an address with your project lets the platform index its activity, so you can list its transactions and receive webhooks for it.

```
curl -X POST https://api.chainforge.example/v1/accounts \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
    "label": "alice-test",
    "metadata": { "customerId": "cust_1042" }
  }'
```

```
{
  "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "label": "alice-test",
  "type": "eoa",
  "metadata": { "customerId": "cust_1042" },
  "createdAt": "2026-10-18T09:12:44Z"
}
```

`metadata` is a free-form object (up to 16 keys) that lets you link an address to your own records. Do not place personal data or secrets in it.

Validate addresses before use

Addresses can be written in all lowercase, or in **mixed case with a checksum** that detects typing errors. The SDKs verify the checksum for you:

```
import { isAddress, toChecksumAddress } from "@chainforge/sdk";

if (!isAddress(userInput)) throw new Error("Invalid address");
const safe = toChecksumAddress(userInput);
```

Always validate user-entered addresses on both the client and the server. A single wrong character sends funds to an address nobody controls, with no recovery.

5.2 Check Balances

```
curl
https://api.chainforge.example/v1/accounts/0x71C7656EC7ab88b098defB751B7401B5
f6d8976F/balance \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
{
  "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "balance": "94999937000000000000",
  "balanceFormatted": "94.999937",
  "symbol": "tFORGE",
  "blockNumber": 4820141,
  "nonce": 1
}
```

- `balance` is a **string** containing an integer number of sparks. It is a string because the value can exceed what JavaScript numbers represent exactly.
- `balanceFormatted` is for display only. Never do arithmetic on it.
- `blockNumber` tells you which block the balance was read at.
- `nonce` is the number of transactions the account has sent, which is also the nonce of its next transaction.

Reading at a specific point

Query parameter	Meaning
`block=latest` (default)	The most recent included block
`block=finalized`	The most recent finalized block. Use for accounting.
`block=<number>`	Historical state at a past block

Working with amounts safely

```
import { parseForge, formatForge } from "@chainforge/sdk";

const sparks = parseForge("1.25");     // 1250000000000000000n (a BigInt)
const text   = formatForge(sparks);    // "1.25"

// Correct: integer arithmetic
const total = parseForge("1.25") + parseForge("0.75");

// WRONG: floating-point arithmetic loses precision
const bad = 1.1 + 2.2; // 3.3000000000000003
```

```
from decimal import Decimal
from chainforge import parse_forge, format_forge

sparks = parse_forge(Decimal("1.25"))   # int
text = format_forge(sparks)
```

> **Warning:** Never use floating-point numbers for money. Keep amounts as integer sparks (or `Decimal` in Python) from the moment they enter your system until you display them.

Listing multiple balances

```
curl "https://api.chainforge.example/v1/accounts/balances?
addresses=0x71C7...976F,0x2B5A...D6cF" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

Batch up to 50 addresses per request to stay within rate limits.

5.3 Send and Track Transactions

The recommended flow

1. **Prepare**: ask the API to fill in the nonce, gas limit and fee suggestions.
2. **Sign**: your code or KMS signs the prepared transaction.
3. **Submit** with an **idempotency key**.
4. **Track** the transaction until it is `finalized` (or fails).

```
import { ChainForge, Wallet, parseForge } from "@chainforge/sdk";
import { randomUUID } from "node:crypto";

const client = new ChainForge({ apiKey: process.env.CHAINFORGE_API_KEY });
const wallet = await Wallet.fromKeystore("alice",
process.env.KEYSTORE_PASSWORD);

const unsigned = await client.transactions.prepare({
  from: wallet.address,
  to: "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  value: parseForge("2"),
  feeSpeed: "normal",           // slow | normal | fast
});

const signed = await wallet.sign(unsigned);

const tx = await client.transactions.submitSigned(signed, {
  idempotencyKey: randomUUID(),  // store this with your order record
});
```

Idempotency

Networks fail. If your request times out you do not know whether the transaction was accepted. If you simply retry, you might pay twice. The `Idempotency-Key` header solves this:

- Send a unique key (a UUID) with each logical operation.
- If you retry with the **same key**, the API returns the original result rather than creating a second transaction.
- Keys are remembered for 24 hours.
- Reusing a key with a **different** payload returns `409 idempotency_conflict`.

Generate the key when you create the business record (for example, the order), store it, and reuse it on every retry of that operation.

Tracking a transaction

Option 1: Poll the transaction

```
curl https://api.chainforge.example/v1/transactions/txn_01JC7X3M9Q \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
{
  "id": "txn_01JC7X3M9Q",
  "hash":
"0x9f3c5d8a1b7e44c2a09d3f6b8e1c2d4a5b6c7d8e9f00112233445566778899aa",
  "status": "finalized",
  "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  "value": "2000000000000000000",
  "nonce": 1,
  "gasUsed": "21000",
  "effectiveGasPrice": "3000000000",
  "fee": "63000000000000",
  "blockNumber": 4820141,
  "blockHash": "0xa1b2...c3d4",
  "submittedAt": "2026-10-18T09:30:12Z",
  "includedAt": "2026-10-18T09:30:14Z",
  "finalizedAt": "2026-10-18T09:30:16Z"
}
```

Option 2: Wait in the SDK

```
const receipt = await tx.waitForFinality({ timeoutMs: 30_000 });
if (receipt.status === "failed") {
  console.error("Reverted:", receipt.revertReason);
}
```

Option 3: Use a webhook (best for production). Subscribe to `transaction.finalized` and `transaction.failed` (chapter 8). Your server is notified without polling.

Polling guidance

If you must poll, do it politely: start at one-second intervals, back off to a maximum of five seconds and stop after a reasonable timeout (60 seconds is generous for a transaction that normally finalizes in about 4 seconds).

Sending many transactions from one account

Because each transaction needs a sequential nonce, sending in parallel requires care.

Approach	How	Trade-off
Sequential	Wait for each to be included before sending the next	Simple but slow
Managed nonces	Fetch the pending nonce once, then increment locally for each transaction	Fast; you must recover if one fails
Multiple sender accounts	Spread load across several wallets	Scales best; more funding logistics

```
let nonce = await client.accounts.getNonce(wallet.address, { block: "pending"
});

for (const payout of payouts) {
  const unsigned = await client.transactions.prepare({
    from: wallet.address,
    to: payout.address,
    value: payout.sparks,
    nonce: nonce++,
  });
  const signed = await wallet.sign(unsigned);
  await client.transactions.submitSigned(signed, { idempotencyKey: payout.id
});
}
```

If transaction N fails to be accepted, transactions N+1 and later will wait in the mempool until the gap is filled. See section 5.5.

Listing transactions

```
curl "https://api.chainforge.example/v1/accounts/0x71C7...976F/transactions?
status=finalized&limit=25" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

List endpoints are **cursor paginated**. Each response contains `data` and a `nextCursor`; pass `cursor=<value>` to get the next page until `nextCursor` is `null`.

5.4 Estimate Gas

Why estimate

Gas limits that are too low make the transaction fail and waste fees. Limits that are too high tie up balance (the excess is refunded, but the account must hold enough FORGE to cover the maximum). Estimates give you a sound starting point.

Estimating a transfer or contract call

```
const estimate = await client.gas.estimate({
  from: wallet.address,
  to: tokenContractAddress,
  data: encodedTransferCall,
});

console.log(estimate.gasLimit);          // e.g. 51420
console.log(estimate.suggested.normal);  // { maxFeePerGas,
maxPriorityFeePerGas }
```

If the call would revert, estimation fails and returns the revert reason. That makes estimation a cheap way to pre-validate a call before you spend any gas.

```
{
  "error": {
    "code": "execution_reverted",
    "message": "Estimation failed because the call would revert.",
    "details": { "revertReason": "ERC20: transfer amount exceeds balance" },
    "requestId": "req_3NQ8R2"
  }
}
```

Tips

1. **Add a buffer** of 10 to 20 percent to estimated `gasLimit` for contract calls.
2. **Re-estimate close to signing.** Gas prices move with demand.
3. **Set a fee ceiling.** Choose `maxFeePerGas` that you are willing to pay in the worst case.
4. **Choose speed to match urgency.** Use `slow` for non-urgent batch work, `fast` when a user is waiting.

5. **Check the sender's balance** covers `value + gasLimit x maxFeePerGas` before you submit.

5.5 Handle Pending, Failed and Replaced Transactions

Pending for too long

A transaction stays `pending` when its fee is lower than what validators are currently accepting, or when an earlier nonce is missing.

Diagnose

1. Fetch the transaction and note its `maxFeePerGas` and `nonce`.
2. Fetch the current `baseFee` from `/v1/gas/estimate`. If `maxFeePerGas` is below it, the transaction cannot be included until the base fee falls.
3. Fetch the account's confirmed nonce. If it is lower than your pending nonce, an earlier transaction is missing or stuck.

Fix: speed up (replace by fee)

Send a new transaction with the **same nonce** and a higher fee. Both `maxFeePerGas` and `maxPriorityFeePerGas` must be at least 10 percent higher than the original. The network keeps the higher-fee one and marks the original `replaced`.

```
const sped = await client.transactions.speedUp(tx.id, { feeSpeed: "fast",
signer: wallet });
```

Fix: cancel

Send a zero-value transaction to your own address, with the same nonce and a higher fee. If it is included first, the original can never execute.

```
const cancel = await client.transactions.cancel(tx.id, { signer: wallet });
```

Failed (reverted) transactions

A `failed` transaction was executed, hit an error and reverted all of its changes. You still paid for the gas it used.

```
{
  "id": "txn_01JC7Y9H4B",
  "status": "failed",
  "revertReason": "Insufficient allowance",
  "gasUsed": "38211",
  "fee": "1146330000000000"
}
```

Common causes of revert:

Cause	Typical `revertReason`	Fix
Insufficient token balance	`transfer amount exceeds balance`	Check balance first
Missing approval	`Insufficient allowance`	Call `approve` before `transferFrom`
Failed contract condition	Contract-specific text	Read the contract's `require` statements
Out of gas	`out_of_gas`	Increase `gasLimit`
Caller not authorized	`Ownable: caller is not the owner`	Use the correct account

To decode custom errors, provide the contract's ABI to the SDK (section 6.4).

Dropped transactions

A `dropped` transaction left the mempool without being included, usually because its fee fell below the base fee for too long or it expired after 30 minutes. Nothing was charged. Recreate it with a fresh fee estimate and the same idempotency key **only if** you confirmed it was not included; otherwise the API returns the original dropped record.

Replaced transactions

The `replaced` status carries a link to the replacement:

```
{
  "id": "txn_01JC7X3M9Q",
  "status": "replaced",
  "replacedBy": "txn_01JC7ZB2WE"
}
```

Follow `replacedBy` to find the transaction that actually executed.

A reliable state machine for your own records

Your order state	Trigger	Next state
`payment_pending`	Transaction submitted	`payment_confirming`
`payment_confirming`	`transaction.finalized` with status success	`paid`
`payment_confirming`	`transaction.failed`	`payment_failed`
`payment_confirming`	`transaction.dropped` or timeout	`payment_retry`
`payment_retry`	Resubmitted	`payment_confirming`

Drive these transitions from webhooks and verify against the API before any irreversible step, such as releasing goods.

---

6. Smart Contracts

Smart contracts let you put business logic on the chain. This chapter walks through the entire contract lifecycle on ChainForge: writing, compiling, deploying, calling, observing events, upgrading and testing.

6.1 Contract Lifecycle Overview

```
Write (Solidity) --> Compile (bytecode + ABI) --> Test (devnet) --> Deploy
(Ember)
      --> Verify source --> Interact (calls, events) --> Monitor --> Upgrade
or retire
```

Stage	Output	Tool
Write	`.sol` source files	Any editor
Compile	Bytecode and **ABI** (the interface description)	`forge compile` (or Hardhat/Foundry)
Test	Passing automated tests	`forge test` on the local devnet
Deploy	A contract address	`forge deploy` or SDK
Verify	Public source code linked to the address	`forge verify`
Interact	Transactions and read calls	API, SDK or explorer
Monitor	Logs, alerts	Webhooks, dashboards

> **Note:** ChainForge is EVM-compatible, so Hardhat, Foundry and other standard Ethereum tooling also work. This guide uses the `forge` CLI so the steps are the same on every machine.

6.2 Write a Contract

This example is a simple **loyalty points** contract. The owner can issue points to customers, customers can transfer points and anyone can read balances.

Create `contracts/LoyaltyPoints.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract LoyaltyPoints {
    string public name = "Acme Loyalty Points";
    string public symbol = "ALP";
    uint8 public constant decimals = 0;

    address public owner;
    uint256 public totalSupply;
    mapping(address => uint256) private balances;

    event PointsIssued(address indexed to, uint256 amount);
    event PointsTransferred(address indexed from, address indexed to, uint256
```

```
amount);

    error NotOwner();
    error InsufficientPoints(uint256 available, uint256 required);
    error ZeroAddress();

    modifier onlyOwner() {
        if (msg.sender != owner) revert NotOwner();
        _;
    }

    constructor() {
        owner = msg.sender;
    }

    function balanceOf(address account) external view returns (uint256) {
        return balances[account];
    }

    function issue(address to, uint256 amount) external onlyOwner {
        if (to == address(0)) revert ZeroAddress();
        balances[to] += amount;
        totalSupply += amount;
        emit PointsIssued(to, amount);
    }

    function transfer(address to, uint256 amount) external returns (bool) {
        if (to == address(0)) revert ZeroAddress();
        uint256 available = balances[msg.sender];
        if (available < amount) revert InsufficientPoints(available, amount);

        balances[msg.sender] = available - amount;
        balances[to] += amount;
        emit PointsTransferred(msg.sender, to, amount);
        return true;
    }
}
```

What each part does

Element	Purpose
`pragma solidity`	Restricts the compiler version so the code behaves as tested
State variables (`owner` , `totalSupply` , `balances`)	Persistent storage; changing them costs gas
`event`	A log that outside programs can subscribe to
`error`	A cheap, structured way to explain a revert
`modifier onlyOwner`	Reusable access check
`constructor`	Runs once at deployment and sets the owner
`view` function	Reads state without a transaction or gas
`emit`	Writes an event to the transaction's log

Safe-coding habits

1. **Check inputs first** and revert early, as the `ZeroAddress` check does.
2. **Update state before external calls** to avoid reentrancy problems. This contract makes no external calls, but the habit matters.
3. **Restrict privileged functions** (`onlyOwner`) and keep the number of privileged roles small.
4. **Prefer well-tested libraries** such as audited token standards instead of writing your own from scratch.
5. **Emit events** for every meaningful state change. Off-chain systems depend on them.

6.3 Compile and Deploy

Initialize and compile

```
forge init loyalty-project
cd loyalty-project
# place LoyaltyPoints.sol in ./contracts
forge compile
```

Compilation writes artifacts to `./artifacts/LoyaltyPoints.json`, which contains the **bytecode** (what runs on the chain) and the **ABI** (the list of functions, events and errors, in JSON). Keep the ABI. Clients need it to talk to the contract.

Deploy to Ember with the CLI

```
forge deploy contracts/LoyaltyPoints.sol \
  --contract LoyaltyPoints \
  --network ember \
  --from alice
```

```
Compiling... done
Estimated gas: 612,408
Submitting deployment transaction: 0x77aa...c01e
Finalized in block 4821377
Contract deployed at: 0x5FbDB2315678afecb367f032d93F642f64180aa3
```

Deploy with the SDK

```
import fs from "node:fs";
import { ChainForge, Wallet } from "@chainforge/sdk";

const client = new ChainForge({ apiKey: process.env.CHAINFORGE_API_KEY });
const wallet = await Wallet.fromKeystore("alice",
process.env.KEYSTORE_PASSWORD);
const artifact = JSON.parse(fs.readFileSync("./artifacts/LoyaltyPoints.json",
"utf8"));

const deployment = await client.contracts.deploy({
  abi: artifact.abi,
  bytecode: artifact.bytecode,
  constructorArgs: [],
  signer: wallet,
});

const receipt = await deployment.waitForFinality();
console.log("Deployed at", receipt.contractAddress);
```

Register and verify the contract

Registering the contract (with its ABI) lets the platform decode its events and calls. **Verifying** publishes the source so anyone can check that the deployed bytecode matches it.

```
forge contract register 0x5FbDB2315678afecb367f032d93F642f64180aa3 \
  --abi artifacts/LoyaltyPoints.abi.json --name "Acme Loyalty Points"

forge verify 0x5FbDB2315678afecb367f032d93F642f64180aa3 \
  --source contracts/LoyaltyPoints.sol --contract LoyaltyPoints --network
ember
```

> **Note:** Record the deployment address, the transaction hash, the compiler version and the git commit in your release notes. You will need them for audits and incident response.

6.4 Call and Query Contracts

Reading data (no gas, no signature)

```
curl -X POST
https://api.chainforge.example/v1/contracts/0x5FbDB2315678afecb367f032d93F642
f64180aa3/call \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "function": "balanceOf",
    "args": ["0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF"]
  }'
```

```
{
  "function": "balanceOf",
  "result": "250",
  "blockNumber": 4821390
}
```

Because the contract is registered with its ABI, you pass the function name and arguments in plain form. The API encodes the call for you.

```
const points = client.contracts.at(contractAddress, abi);
const balance = await points.read.balanceOf([customerAddress]);
```

Writing data (a transaction)

```
const tx = await points.write.issue([customerAddress, 100n], { signer: wallet
});
const receipt = await tx.waitForFinality();

for (const log of receipt.events) {
  if (log.name === "PointsIssued") {
    console.log("Issued", log.args.amount, "to", log.args.to);
  }
}
```

```
contract = client.contracts.at(contract_address, abi)
tx = contract.write.issue([customer_address, 100], signer=wallet)
receipt = tx.wait_for_finality()
for log in receipt.events:
    if log.name == "PointsIssued":
        print("Issued", log.args["amount"], "to", log.args["to"])
```

Simulating before you send

A **simulation** runs the call against current state without committing it. It reveals reverts and returns what the call would have changed.

```
const sim = await points.simulate.transfer([friendAddress, 10_000n], { from:
wallet.address });
if (!sim.success) console.log("Would revert:", sim.revertReason);
```

Decoding errors

When you supply the ABI, custom errors are decoded into readable form:

```
{
  "status": "failed",
  "revertReason": "InsufficientPoints(available=250, required=10000)"
}
```

6.5 Events and Logs

Contracts cannot push data to your servers, and reading storage for changes is inefficient. **Events** solve this: when a contract emits an event, the data is written to the transaction's **log**, which is cheap to store and easy to search.

Anatomy of a log

```
{
  "address": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  "event": "PointsIssued",
  "args": { "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF", "amount":
"100" },
  "blockNumber": 4821391,
  "transactionHash": "0xb7c1...91ee",
  "logIndex": 0
}
```

Field	Meaning
`address`	The contract that emitted the event
`event` and `args`	The event name and decoded parameters
`blockNumber`, `transactionHash`	Where it happened
`logIndex`	Position of the log within the block, used to order and de-duplicate

Querying past events

```
curl "https://api.chainforge.example/v1/contracts/0x5FbD...0aa3/events?
event=PointsIssued&fromBlock=4821000&toBlock=latest&limit=100" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

Indexed parameters

Parameters declared `indexed` in Solidity can be used as filters, for example `to=0x2B5A...`. Index the fields you will search on, such as account addresses and IDs. Do not index large data.

For live delivery, see chapter 8.

6.6 Upgrading Contracts

Deployed code is immutable. If you need to change behavior later, you must plan for it **before** deployment. There are three common strategies.

Strategy	How it works	Pros	Cons
Redeploy and migrate	Deploy a new contract, then move state and point users to it	Simple, no special design needed	Address changes, migration can be costly
Proxy pattern	Users call a permanent proxy that forwards calls to a replaceable implementation contract	Keeps address and state	More complex; admin key becomes a high-value target
Modular design	Small, single-purpose contracts connected through a registry	Replace only what changes	Needs careful interface design

Proxy pattern in brief

```
User --> Proxy (stores data, fixed address)
            |  delegatecall
            v
        Implementation v1  --(upgrade)-->  Implementation v2
```

The proxy holds the state. The implementation holds the logic. An upgrade changes which implementation the proxy points to.

Upgrade safety rules

1. **Never reorder or remove existing state variables** in a new implementation. Only append new ones, because the proxy's storage layout must stay compatible.
2. **Protect the upgrade function** with a multi-signature wallet and, ideally, a **time-lock** (a delay between announcing and executing an upgrade) so users can review.
3. **Use audited proxy libraries** rather than writing your own.

4. **Test the upgrade on Ember** with real data copied from a representative state before you do it on mainnet.
5. **Announce upgrades** in your changelog and communicate with integrators in advance.

> **Warning:** Whoever can upgrade a contract can change what it does. If your users trust the contract because it is immutable, adding an upgrade path changes that promise. Document who holds upgrade rights and under what conditions they are used.

6.7 Testing Contracts

The testing pyramid

Level	What it covers	Where it runs	Speed
Unit tests	Each function in isolation	Local devnet	Seconds
Integration tests	Contract plus your application code	Local devnet or Ember	Minutes
Fuzz and property tests	Random inputs checked against rules that must always hold	Local devnet	Minutes
Audit	Independent expert review	Offline	Weeks

Start the local devnet

```
forge devnet start --accounts 10 --balance 1000
# Chain ID: 7702
# RPC: http://127.0.0.1:8545
# 10 accounts funded with 1000 dFORGE each
```

A unit test

Create `test/LoyaltyPoints.test.js`:

```
import { describe, it, expect, beforeEach } from "@chainforge/test";

describe("LoyaltyPoints", () => {
  let points, owner, alice, bob;

  beforeEach(async ({ deploy, accounts }) => {
    [owner, alice, bob] = accounts;
    points = await deploy("LoyaltyPoints", { from: owner });
  });

  it("lets the owner issue points", async () => {
    await points.write.issue([alice.address, 100n], { from: owner });
    expect(await points.read.balanceOf([alice.address])).toBe(100n);
  });

  it("rejects issuing by non-owners", async () => {
    await expect(
      points.write.issue([alice.address, 100n], { from: bob })
    ).toRevertWith("NotOwner");
  });

  it("transfers points", async () => {
    await points.write.issue([alice.address, 100n], { from: owner });
    await points.write.transfer([bob.address, 40n], { from: alice });
    expect(await points.read.balanceOf([bob.address])).toBe(40n);
    expect(await points.read.balanceOf([alice.address])).toBe(60n);
  });

  it("rejects overdrafts", async () => {
    await points.write.issue([alice.address, 10n], { from: owner });
    await expect(
      points.write.transfer([bob.address, 11n], { from: alice })
    ).toRevertWith("InsufficientPoints");
  });

  it("emits PointsIssued", async () => {
    const tx = await points.write.issue([alice.address, 5n], { from: owner
});
    const receipt = await tx.wait();
    expect(receipt.events[0].name).toBe("PointsIssued");
  });
});
```

```
forge test
```

What to test

- **Every public function**, with both valid and invalid inputs.
- **Access control**: each privileged function must reject unauthorized callers.
- **Boundaries**: zero amounts, maximum values, empty arrays.
- **State invariants**: for example, the sum of all balances always equals `totalSupply`.
- **Events**: confirm each emits the right event with the right arguments.
- **Gas usage** of key functions, so regressions are caught.

A property test (invariant)

```
it("keeps total supply equal to the sum of balances", async () => {
  await fuzz({ runs: 500 }, async (rand) => {
    const who = rand.pick(accounts);
    const amount = rand.int(0, 1_000);
    await points.write.issue([who.address, BigInt(amount)], { from: owner });
    const sum = (await Promise.all(accounts.map(a =>
points.read.balanceOf([a.address]))))
      .reduce((x, y) => x + y, 0n);
    expect(sum).toBe(await points.read.totalSupply());
  });
});
```

Before mainnet

1. All tests pass on the local devnet.
2. The same flows are exercised end to end on Ember.
3. Test coverage of critical paths is reviewed.
4. An independent audit is completed for contracts that hold meaningful value, and every finding is fixed or formally accepted.
5. A rollback and pause plan is in place.

---

7. Tokens

Tokens are digital assets that live in smart contracts and represent value, access or ownership. ChainForge supports two standard token types, both compatible with the widely used Ethereum token standards, so existing wallets and tools recognize them.

Standard	Family	Each unit is…	Examples
CFT-20 (ERC-20 compatible)	Fungible	Interchangeable with every other unit	Stablecoins, loyalty points, utility tokens
CFT-721 (ERC-721 compatible)	Non-fungible	Unique	Tickets, collectibles, property deeds, certificates

> **Note:** The native token FORGE is **not** a contract. It is built into the protocol and is used for gas and staking. CFT tokens are contracts that you or others deploy.

7.1 Fungible Tokens

A fungible token is a ledger kept inside a contract: a mapping from addresses to balances, with rules for moving units between them.

The standard interface

Every CFT-20 token exposes the same functions, which is what lets any wallet or exchange work with it:

Function	Kind	Purpose
`name()`, `symbol()`, `decimals()`	Read	Display information
`totalSupply()`	Read	Total units in existence
`balanceOf(owner)`	Read	Units held by an address
`transfer(to, amount)`	Write	Move units from the caller to another address
`approve(spender, amount)`	Write	Allow another address to spend up to `amount` of the caller's tokens

Function	Kind	Purpose
`allowance(owner, spender)`	Read	Remaining amount a spender may use
`transferFrom(from, to, amount)`	Write	Move units on someone's behalf, using an approval

Two events are emitted: `Transfer(from, to, value)` and `Approval(owner, spender, value)`.

Decimals

Token balances are stored as integers. The `decimals` value says how many places to shift for display, exactly as sparks relate to FORGE.

Token	`decimals`	Stored value	Displayed value
Loyalty points	0	`250`	250 points
Payment token (USD-pegged)	6	`12500000`	12.50
Utility token	18	`1000000000000000000`	1.0

Always read `decimals()` from the contract rather than assuming it.

Creating a token with the platform's template

You can deploy an audited token template without writing Solidity:

```
forge token create \
  --type cft20 \
  --name "Acme Dollar" \
  --symbol ACMD \
  --decimals 6 \
  --initial-supply 1000000 \
  --mintable true \
  --burnable true \
  --pausable true \
  --network ember --from alice
```

```
Token deployed at: 0x8464135c8F25Da09e49BC8782676a84730C318bC
Initial supply of 1,000,000.000000 ACMD minted to 0x71C7...976F
```

Or through the REST API:

```
curl -X POST https://api.chainforge.example/v1/tokens \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: 31d9c0aa-52f4-4f08-9d2e-18b6c5a1e7b3" \
  -d '{
    "type": "cft20",
    "name": "Acme Dollar",
    "symbol": "ACMD",
    "decimals": 6,
    "initialSupply": "1000000000000",
    "features": { "mintable": true, "burnable": true, "pausable": true },
    "owner": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F"
  }'
```

Note that `initialSupply` is expressed in the token's smallest unit: 1,000,000 tokens with 6 decimals is `1000000 x 10^6 = 1000000000000`.

Reading balances and supply

```
curl
https://api.chainforge.example/v1/tokens/0x8464135c8F25Da09e49BC8782676a84730
C318bC/balances/0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
{
  "token": "0x8464135c8F25Da09e49BC8782676a84730C318bC",
  "holder": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  "balance": "12500000",
  "balanceFormatted": "12.5",
  "symbol": "ACMD",
  "decimals": 6
}
```

Features to choose deliberately

Feature	What it enables	Risk to weigh
Mintable	The owner can create new units	Inflation; the owner key is powerful

Feature	What it enables	Risk to weigh
Burnable	Holders can destroy their own units	Low
Pausable	The owner can freeze all transfers in an emergency	Centralization; users must trust the owner
Capped supply	Hard maximum that minting cannot exceed	Cannot be changed later
Allow/deny lists	Restrict who can hold or move the token	Regulatory and compliance use; adds administration

The approve and transferFrom pattern

Some actions, such as paying a shop contract, need the contract to pull tokens from a user. Because a contract cannot take tokens without permission, the user first **approves** it:

```
// 1. The user allows the shop to spend up to 50 ACMD
await token.write.approve([shopAddress, 50_000_000n], { signer: userWallet
});

// 2. The shop contract later calls token.transferFrom(user, shop, amount)
await shop.write.pay([orderId], { signer: userWallet });
```

> **Warning:** Avoid unlimited approvals. A common convenience is to approve the maximum possible amount, which means a bug or compromise in the approved contract can drain the user's entire balance. Approve only what is needed for the transaction.

7.2 Non-fungible Tokens (NFTs)

In an NFT contract, each token has a unique **token ID** and exactly one owner. Two tokens from the same contract are never interchangeable.

The standard interface

Function	Purpose
`ownerOf(tokenId)`	Who owns this token
`balanceOf(owner)`	How many tokens an address owns
`transferFrom(from, to, tokenId)`	Move a token
`safeTransferFrom(from, to, tokenId)`	Move a token, checking that a receiving contract can handle it
`approve(to, tokenId)`	Allow one address to move one token
`setApprovalForAll(operator, approved)`	Allow an operator to move all of an owner's tokens
`tokenURI(tokenId)`	Link to the token's metadata

Metadata

The token itself stores only an ID and an owner. Descriptive content such as name, image and attributes lives in **metadata**, a JSON file that `tokenURI` points to:

```
{
  "name": "Acme Conference 2026 - Early Bird Ticket #0042",
  "description": "Admits one person to Acme Conference 2026, Bengaluru.",
  "image":
"ipfs://bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqabf3oclgtqy55fbzdi",
  "attributes": [
    { "trait_type": "Tier", "value": "Early Bird" },
    { "trait_type": "Seat", "value": "A-12" },
    { "trait_type": "Event Date", "display_type": "date", "value": 1795046400
}
  ]
}
```

Where to store metadata

Storage	Durability	Mutable?	Notes
Content-addressed storage (IPFS-style)	High, if pinned	No: changing the content changes the address	Preferred for permanent assets
Your own HTTPS server	Depends on you	Yes	Simple but a single point of failure; if the server goes away the token loses its meaning
On-chain	Highest	Depends on contract	Expensive; suited to small data

Deploying and minting

```
forge token create \
  --type cft721 \
  --name "Acme Conference Tickets" \
  --symbol ACT \
  --base-uri "ipfs://bafybeih5kq.../" \
  --network ember --from alice
```

```
const tickets = client.contracts.at(ticketAddress, cft721Abi);

const tx = await tickets.write.mint(
  [attendeeAddress, 42n, "ipfs://bafybeih5kq.../42.json"],
  { signer: organizerWallet }
);
await tx.waitForFinality();

console.log("Owner of #42:", await tickets.read.ownerOf([42n]));
```

Listing a wallet's NFTs

```
curl "https://api.chainforge.example/v1/accounts/0x2B5A...D6cF/nfts?
collection=0x9A67...41b2&limit=50" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
{
  "data": [
    {
      "collection": "0x9A676e781A523b5d0C0e43731313A708CB607508",
      "tokenId": "42",
      "name": "Acme Conference 2026 - Early Bird Ticket #0042",
      "image":
"ipfs://bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqabf3oclgtqy55fbzdi",
      "metadataStatus": "ok"
    }
  ],
  "nextCursor": null
}
```


7.3 Minting, Transferring and Burning

Minting

Minting creates new tokens. Only addresses with the minter role can do it.

Token type	Mint call	Result
CFT-20	`mint(to, amount)`	`amount` new units added to `to`; `totalSupply` increases
CFT-721	`mint(to, tokenId, uri)`	A new unique token owned by `to`

```
// Fungible
await token.write.mint([customerAddress, 5_000_000n], { signer: minterWallet
});

// Batch mint to many recipients (one transaction)
await token.write.batchMint([recipients, amounts], { signer: minterWallet });
```

Mint safely

1. Hold minter keys in a KMS or multi-signature wallet.
2. Rate-limit minting in your own backend, and log who requested each mint.

3. Reconcile `totalSupply` against your own records daily.
4. For capped tokens, check the remaining headroom before submitting.

Transferring

```
// Fungible: send 2.50 ACMD (6 decimals)
await token.write.transfer([recipientAddress, 2_500_000n], { signer:
senderWallet });

// NFT: send ticket #42
await tickets.write.safeTransferFrom(
  [senderAddress, recipientAddress, 42n],
  { signer: senderWallet }
);
```

Before every transfer, **simulate** it (section 6.4) and **validate the recipient address** (section 5.1). Sending tokens to a contract that cannot handle them, or to a mistyped address, usually loses them permanently.

Burning

Burning permanently destroys tokens and reduces supply.

```
await token.write.burn([1_000_000n], { signer: holderWallet });   // CFT-20
await tickets.write.burn([42n], { signer: holderWallet });         // CFT-721
```

Common uses: redeeming points, consuming a one-time ticket, retiring a certificate or reducing supply. Burn is **irreversible**, so confirm with the user before executing.

Tracking token activity

Every mint is a `Transfer` event **from the zero address**, and every burn is a `Transfer` event **to the zero address**. This means a single event stream describes the whole life of a token:

Action	`from`	`to`
Mint	`0x000...000`	Recipient
Transfer	Sender	Recipient
Burn	Holder	`0x000...000`

---

8. Events and Webhooks

Real-time applications need to know when something happens on-chain: a payment arrives, a token is minted, a transaction finalizes. This chapter shows how to receive that information reliably.

8.1 Subscribing to On-Chain Events

There are three ways to learn about events. Choose based on where your code runs.

Method	How it works	Best for	Caveat
Polling	Your code repeatedly asks the API for new data	Scripts, low volume	Wasteful, adds delay, uses rate limit
WebSocket subscription	A long-lived connection streams events to your process	Dashboards, bots, services that stay running	Connection can drop; you must reconnect and catch up
Webhooks	The platform sends an HTTPS request to your server	Backend systems, serverless functions	Your endpoint must be public and verify requests

WebSocket subscriptions

```
import { ChainForge } from "@chainforge/sdk";

const client = new ChainForge({ apiKey: process.env.CHAINFORGE_API_KEY });

const sub = client.subscribe({
  type: "contract.event",
  contract: "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  event: "PointsIssued",
  finality: "finalized",
});

sub.on("event", (e) => {
  console.log(`Issued ${e.args.amount} to ${e.args.to} in block
${e.blockNumber}`);
});

sub.on("error", (err) => console.error("Subscription error", err));
sub.on("reconnect", ({ resumedFromBlock }) => console.log("Resumed from",
resumedFromBlock));
```

```
sub = client.subscribe(
    type="contract.event",
    contract="0x5FbDB2315678afecb367f032d93F642f64180aa3",
    event="PointsIssued",
    finality="finalized",
)

@sub.on("event")
def handle(e):
    print(f"Issued {e.args['amount']} to {e.args['to']} in block
{e.block_number}")

sub.run_forever()
```

Subscription types

`type`	Fires when	Useful filters
`block.new`	A new block is produced or finalized	`finality`
`transaction.status`	A watched transaction changes status	`transactionId` or `hash`
`account.activity`	A watched address sends or receives FORGE	`address`, `direction`
`contract.event`	A contract emits a log	`contract`, `event`, indexed arguments
`token.transfer`	A token moves	`token`, `from`, `to`

The `finality` option

- `included` delivers events as soon as they are in a block (faster, but in rare cases of reordering a delivered event may not become final).
- `finalized` waits for finality (about 2 seconds longer, and never reversed).

Default to `finalized` for anything that changes money or inventory.

Handling disconnects

Connections drop. A correct consumer:

1. **Reconnects automatically** with backoff (the SDK does this).
2. **Resumes from the last processed block** rather than from "now", so no events are lost during the gap.
3. **De-duplicates** using the pair (`transactionHash`, `logIndex`), because an event can be delivered more than once around a reconnect.
4. **Processes in order** within a block when order matters.

8.2 Configuring Webhooks

A **webhook** is an HTTPS endpoint on your server that ChainForge calls when a subscribed event happens. You do not need to hold a connection open or poll.

Step 1: Prepare an endpoint

Your endpoint must:

- Use **HTTPS** with a valid certificate (plain HTTP is rejected for live webhooks).
- Accept `POST` requests with a JSON body.
- Respond with a `2xx` status **within 10 seconds**.
- Do heavy work asynchronously: acknowledge first, process afterwards via a queue.

Step 2: Register the webhook

In the Console: Project → Webhooks → **Add endpoint**, then enter the URL and choose events.

Or through the API:

```
curl -X POST https://api.chainforge.example/v1/webhooks \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://shop.acme.example/hooks/chainforge",
    "description": "Order payment notifications",
    "events": ["transaction.finalized", "transaction.failed",
"token.transfer"],
    "filters": {
      "addresses": ["0x71C7656EC7ab88b098defB751B7401B5f6d8976F"]
    },
    "enabled": true
  }'
```

```
{
  "id": "wh_01JC9K4TQZ",
  "url": "https://shop.acme.example/hooks/chainforge",
  "events": ["transaction.finalized", "transaction.failed",
"token.transfer"],
  "secret": "whsec_8f3b1c7d9e2a4b6c",
  "enabled": true,
  "createdAt": "2026-10-18T10:05:00Z"
}
```

The `secret` is shown **once**. Store it securely. You will use it to verify that requests really came from ChainForge.

Available webhook events

Event	Sent when
`transaction.pending`	A transaction is accepted into the mempool
`transaction.included`	A transaction is placed in a block
`transaction.finalized`	A transaction reaches finality
`transaction.failed`	A transaction was included but reverted
`transaction.dropped`	A transaction left the mempool without inclusion
`account.received`	A registered address receives FORGE
`token.transfer`	A watched token moves
`contract.event`	A registered contract emits a matching log
`contract.deployed`	A contract deployment finalizes

The delivery payload

```
{
  "id": "evt_01JC9M7N2X",
  "type": "transaction.finalized",
  "apiVersion": "v1",
  "createdAt": "2026-10-18T10:07:41Z",
  "network": "ember",
  "data": {
    "id": "txn_01JC9M6A1B",
    "hash": "0x4e1d...a9c3",
    "status": "finalized",
    "from": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
    "to": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
    "value": "5000000000000000000",
    "blockNumber": 4822095
  }
}
```

Every delivery has a unique `id`. Use it to avoid processing the same event twice.

Testing webhooks

- Use the Console's **Send test event** button, or `forge webhook test wh_01JC9K4TQZ --event transaction.finalized`.
- During development, expose your local server with a tunneling tool and register the tunnel URL on a **testnet** project.
- Review **Delivery logs** in the Console to see each attempt's request, response code and timing.

8.3 Verifying Webhook Signatures

Anyone who discovers your webhook URL could post fake events to it. A forged "payment finalized" message could trick your system into shipping goods for free. ChainForge therefore **signs** every delivery, and you must verify the signature before trusting the content.

How signing works

Each request includes two headers:

Header	Content
`X-ChainForge-Timestamp`	Unix time (seconds) when the request was signed
`X-ChainForge-Signature`	`v1=` followed by a hex HMAC-SHA256 of the string `<timestamp>.<raw request body>` using your webhook secret

Verification steps

1. Read the **raw** request body exactly as received. Do not parse and re-serialize it, because that changes the bytes.
2. Read the timestamp header and reject requests older than **5 minutes** (this stops replay of captured requests).
3. Compute `HMAC-SHA256(secret, timestamp + "." + rawBody)`.
4. Compare it with the header value using a **constant-time** comparison.
5. Only then parse the JSON and process it.

Node.js (Express)

```
import express from "express";
import crypto from "node:crypto";

const app = express();
const SECRET = process.env.CHAINFORGE_WEBHOOK_SECRET;

// Use the raw body for signature checks
app.post("/hooks/chainforge", express.raw({ type: "application/json" }),
(req, res) => {
  const timestamp = req.header("X-ChainForge-Timestamp");
  const signature = req.header("X-ChainForge-Signature") ?? "";

  if (!timestamp || Math.abs(Date.now() / 1000 - Number(timestamp)) > 300) {
    return res.status(400).send("Stale or missing timestamp");
  }

  const expected = "v1=" + crypto
    .createHmac("sha256", SECRET)
    .update(`${timestamp}.${req.body.toString("utf8")}`)
    .digest("hex");

  const ok =
    signature.length === expected.length &&
    crypto.timingSafeEqual(Buffer.from(signature), Buffer.from(expected));

  if (!ok) return res.status(401).send("Invalid signature");

  const event = JSON.parse(req.body.toString("utf8"));
  queue.enqueue(event);          // process asynchronously
  res.status(200).send("ok");    // acknowledge quickly
});
```

Python (Flask)

```
import hmac, hashlib, time, os, json
from flask import Flask, request, abort

app = Flask(__name__)
SECRET = os.environ["CHAINFORGE_WEBHOOK_SECRET"].encode()
```

```
@app.post("/hooks/chainforge")
def hook():
    timestamp = request.headers.get("X-ChainForge-Timestamp", "")
    signature = request.headers.get("X-ChainForge-Signature", "")
    raw = request.get_data()  # bytes, exactly as received

    if not timestamp.isdigit() or abs(time.time() - int(timestamp)) > 300:
        abort(400)

    signed = timestamp.encode() + b"." + raw
    expected = "v1=" + hmac.new(SECRET, signed, hashlib.sha256).hexdigest()

    if not hmac.compare_digest(signature, expected):
        abort(401)

    event = json.loads(raw)
    enqueue(event)
    return "ok", 200
```

The SDKs provide a helper, `client.webhooks.constructEvent(rawBody, headers, secret)`, which performs all of these checks and returns the parsed event.

Rotating secrets

To rotate a secret without downtime, call `POST /v1/webhooks/{id}/rotate-secret`. For 24 hours both the old and new secrets are accepted, and each delivery is signed with both (`v1=` appears twice, comma separated). Update your server during that window, then the old secret expires.

8.4 Retries and Idempotency

Delivery guarantees

ChainForge provides **at-least-once** delivery. Every event is delivered one or more times, never zero (unless you disable the webhook). Duplicates are possible, and events may arrive out of order. Your handler must tolerate both.

Retry schedule

A delivery fails if your endpoint returns a non-`2xx` status, times out or is unreachable. ChainForge then retries with exponential backoff:

Attempt	Delay after previous failure
1	Immediate
2	30 seconds
3	5 minutes
4	30 minutes
5	2 hours
6	6 hours
7	24 hours

After the seventh failed attempt the event is marked **failed** in the delivery log. If an endpoint fails continuously for three days, ChainForge disables it and emails the organization's administrators.

You can replay missed events at any time:

```
curl -X POST
https://api.chainforge.example/v1/webhooks/wh_01JC9K4TQZ/deliveries/evt_01JC9
M7N2X/redeliver \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

Making your handler idempotent

Because duplicates happen, processing an event twice must have the same effect as processing it once.

```
async function handleEvent(event) {
  // Atomically record the event ID; fails if it already exists
  const firstTime = await db.processedEvents.insertIfAbsent(event.id);
  if (!firstTime) return; // already handled

  if (event.type === "transaction.finalized") {
    await db.orders.markPaid({ txHash: event.data.hash });
  }
}
```

Best practice is to make the "record the event ID" step and the business update part of **one database transaction**, so a crash between them cannot lose or duplicate work.

Handling out-of-order events

Do not assume `transaction.included` arrives before `transaction.finalized`. Compare against current state:

- Never move an order **backwards** (for example, from `paid` to `confirming`).
- Use `data.blockNumber` and `createdAt` to ignore stale updates.
- When in doubt, call `GET /v1/transactions/{id}` and treat the API as the source of truth.

Reconciliation

Even a good webhook setup should be backed by a periodic **reconciliation job**. Once an hour, list recent transactions for your accounts and confirm that each one has been processed. This catches events lost to long outages and gives you confidence in your records.

---

9. SDKs and Tools

This chapter is a practical reference to the ChainForge developer toolkit: two official SDKs, the `forge` command-line tool, the local devnet and the block explorer.

9.1 JavaScript/TypeScript SDK

Package: `@chainforge/sdk` · **Requires:** Node.js 18+ (also works in modern browsers for read operations) · **Language:** TypeScript with full type definitions

Install and initialize

```
npm install @chainforge/sdk
```

```
import { ChainForge } from "@chainforge/sdk";

const client = new ChainForge({
  apiKey: process.env.CHAINFORGE_API_KEY!,
  network: "ember",          // "mainnet" | "ember" | "devnet"
  timeoutMs: 15_000,
  maxRetries: 3,
});
```

Client options

Option	Type	Default	Description
`apiKey`	string	required	Your `cfk_test_` or `cfk_live_` key
`network`	string	Inferred from key	Overrides network; must match the key
`baseUrl`	string	Platform URL	Point to a proxy or mock server
`timeoutMs`	number	15000	Per-request timeout
`maxRetries`	number	3	Automatic retries for `429` and `5xx`
`fetch`	function	Global `fetch`	Custom HTTP implementation

Resource modules

Module	Purpose	Example
`client.accounts`	Balances, nonces, registration	`client.accounts.getBalance(address)`
`client.transactions`	Prepare, submit, track	`client.transactions.send({...})`
`client.gas`	Fee estimates	`client.gas.estimate({...})`
`client.contracts`	Deploy, read, write, events	`client.contracts.at(addr, abi)`
`client.tokens`	Create and inspect tokens	`client.tokens.create({...})`

Module	Purpose	Example
`client.webhooks`	Manage endpoints, verify events	`client.webhooks.constructEvent(...)`
`client.subscribe`	Real-time subscriptions	`client.subscribe({ type: "block.new" })`

Error handling

```
import { ChainForgeError, RateLimitError, ValidationError } from
"@chainforge/sdk";

try {
  await client.transactions.send({ from: wallet, to, value });
} catch (err) {
  if (err instanceof RateLimitError) {
    console.warn(`Retry after ${err.retryAfter}s`);
  } else if (err instanceof ValidationError) {
    console.error("Bad input:", err.details);
  } else if (err instanceof ChainForgeError) {
    console.error(err.code, err.message, "requestId:", err.requestId);
  } else {
    throw err;
  }
}
```

Pagination helper

```
for await (const tx of client.accounts.listTransactions(address, { status:
"finalized" })) {
  console.log(tx.hash, tx.value);
}
```

The async iterator follows `nextCursor` automatically.

9.2 Python SDK

Package: `chainforge-sdk` · **Requires:** Python 3.9+ · **Style:** synchronous client plus an asyncio client

```
pip install chainforge-sdk
```

```
import os
from chainforge import ChainForge

client = ChainForge(
    api_key=os.environ["CHAINFORGE_API_KEY"],
    network="ember",
    timeout=15,
    max_retries=3,
)

balance =
client.accounts.get_balance("0x71C7656EC7ab88b098defB751B7401B5f6d8976F")
print(balance.formatted, balance.symbol)
```

Async usage

```
import asyncio
from chainforge import AsyncChainForge

async def main():
    async with AsyncChainForge(api_key=os.environ["CHAINFORGE_API_KEY"]) as
client:
        tx = await client.transactions.send(sender=wallet, to=recipient,
value=10**18)
        receipt = await tx.wait_for_finality()
        print(receipt.status)

asyncio.run(main())
```

Naming conventions

The Python SDK uses `snake_case` for methods and parameters and returns dataclass-style objects. Where the JavaScript SDK uses `from`, a Python reserved word, the Python SDK uses `sender`.

JavaScript	Python
`client.transactions.send({ from, to, value })`	`client.transactions.send(sender=, to=, value=)`
`parseForge("1.5")`	`parse_forge("1.5")`
`tx.waitForFinality()`	`tx.wait_for_finality()`
`ValidationError`	`chainforge.errors.ValidationError`

Pagination

```
for tx in client.accounts.list_transactions(address, status="finalized"):
    print(tx.hash, tx.value)
```

9.3 CLI Reference Summary

The `forge` command-line tool is the quickest way to explore ChainForge, script routine tasks and run a local network. Run `forge --help` or `forge <command> --help` for full options.

Setup and configuration

Command	Description
`forge auth login --api-key <key>`	Store credentials
`forge auth status`	Show the active project and key permissions
`forge config set network <name>`	Select `mainnet`, `ember` or `devnet`
`forge config list`	Show all settings

Wallets and funds

Command	Description
`forge wallet create --name <n>`	Generate an encrypted keystore wallet

Command	Description
`forge wallet list`	List local wallets
`forge wallet import --name <n> --private-key <k>`	Import an existing key (testnet only recommended)
`forge balance <address>`	Show native balance
`forge faucet request --address <a>`	Request test tokens (Ember)
`forge send --from <w> --to <a> --amount <n>`	Send native tokens

Transactions and gas

Command	Description
`forge tx get <hash-or-id>`	Fetch a transaction
`forge tx wait <hash-or-id>`	Wait for finality
`forge tx speed-up <id>`	Replace with a higher fee
`forge tx cancel <id>`	Cancel a pending transaction
`forge gas estimate --to <a> --value <n>`	Estimate gas and fees

Contracts and tokens

Command	Description
`forge init <name>`	Scaffold a project
`forge compile`	Compile contracts
`forge test`	Run tests on the local devnet
`forge deploy <file> --contract <C>`	Deploy a contract
`forge verify <address>`	Publish and verify source
`forge contract register <address> --abi <file>`	Register with the platform

Command	Description
`forge call <address> <function> [args]`	Read-only call
`forge token create --type cft20|cft721 ...`	Deploy a token from a template

Webhooks and tooling

Command	Description
`forge webhook list`	Show endpoints
`forge webhook test <id> --event <type>`	Send a test event
`forge logs tail --contract <a>`	Stream a contract's events in the terminal
`forge devnet start|stop|reset`	Control the local network

Output formats and scripting

Use `--json` for machine-readable output and `--quiet` to print only the primary result, which makes the CLI easy to combine with other tools:

```
HASH=$(forge send --from alice --to 0x2B5A...D6cF --amount 1 --quiet)
forge tx wait "$HASH" --json | jq '.status'
```

Exit codes: `0` success, `1` general error, `2` invalid input, `3` transaction failed or reverted, `4` timeout, `5` authentication problem.

9.4 Local Devnet and Block Explorer

The local devnet

The devnet is a single-node ChainForge network on your computer. It produces blocks instantly, funds test accounts and can be reset to a clean state.

```
forge devnet start --accounts 10 --balance 1000 --block-time 0
```

Option	Description
`--accounts <n>`	Number of pre-funded accounts
`--balance <n>`	dFORGE per account
`--block-time <s>`	Seconds between blocks; `0` produces a block per transaction
`--fork ember`	Start with a copy of the current Ember state, useful for reproducing bugs
`--port <p>`	RPC port (default 8545)

Point the SDK at it:

```
const client = new ChainForge({ network: "devnet", baseUrl:
"http://127.0.0.1:8545" });
```

Useful devnet-only controls

```
forge devnet time-travel --seconds 86400   # jump the clock forward one day
forge devnet mine --blocks 5               # produce blocks on demand
forge devnet snapshot                      # save state, prints a snapshot ID
forge devnet revert --snapshot <id>        # return to a saved state
forge devnet reset                         # wipe everything
```

Snapshots make tests repeatable: take one after setup, run a test, then revert.

The block explorer

The explorer is a web interface for looking at any public network.

Network	URL
Mainnet	`https://explorer.chainforge.example`
Ember	`https://explorer.ember.chainforge.example`

What you can look up

Search for	You will see
Transaction hash	Status, block, sender, recipient, value, gas used, fee, decoded input, event logs, revert reason
Address	Balance, token holdings, transaction history, contract code and verified source (if a contract)
Block number or hash	Timestamp, proposer, transactions, gas used, finality status
Token	Supply, holders, transfers, contract details

Using the explorer for debugging

1. Paste the transaction hash into the search bar.
2. Check **Status** and **Finality**. A `Fail` status shows the revert reason.
3. Open **Logs** to see which events fired, or did not fire.
4. Open **Input data** and decode it with the verified ABI to confirm the call was what you intended.
5. Compare **Gas used** with **Gas limit**. A transaction that used 100 percent of its limit probably ran out of gas.

Verified contracts also provide a **Read** tab and a **Write** tab, so you can call functions directly from the explorer with a connected wallet, which is convenient for quick checks on Ember.

---

10. Tutorials

These three end-to-end tutorials combine the concepts from earlier chapters. Each lists what you will build, what you need, the steps, and how to check that it works. All use the **Ember testnet**, so nothing here involves real value.

10.1 Build a Token Payment Flow

Scenario. Acme Books sells ebooks. A customer pays in ACMD (a USD-pegged test token with 6 decimals). Your server creates an order, shows the customer a payment request, detects the payment on-chain and then releases the download.

You will learn: creating a token, generating unique payment references, listening for token transfers with a webhook, verifying amounts and finality, and handling edge cases such as underpayment and late payment.

Prerequisites: chapters 2, 5, 7 and 8; Node.js 18+; a public HTTPS URL for webhooks (use a tunneling tool during development).

Architecture

```
Customer wallet --(1) transfer ACMD--> Acme deposit address
                                              |
                          ChainForge indexer <-+
                                   |
                          (2) webhook: token.transfer (finalized)
                                   v
                            Acme server --(3) match order, verify amount-->
mark paid, deliver ebook
```

Step 1: Deploy the payment token (Ember only)

```
forge token create --type cft20 --name "Acme Dollar" --symbol ACMD \
  --decimals 6 --initial-supply 1000000 --mintable true \
  --network ember --from alice
# Token deployed at: 0x8464135c8F25Da09e49BC8782676a84730C318bC
```

Give a test customer some ACMD:

```
forge call 0x8464...18bC transfer 0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF
50000000 --from alice
```

Step 2: Choose how to identify payments

A transfer on-chain does not carry an order number. You have two common options:

Approach	How it works	Pros	Cons
Unique deposit address per order	Derive a fresh address for each order from a master key or KMS	Unambiguous matching	More addresses to sweep and manage

Approach	How it works	Pros	Cons
Payment contract with order ID	Customer calls `pay(orderId, amount)` on your contract, which emits an event with the order ID	Clean on-chain record, supports refunds	Needs a contract and an approval step

This tutorial uses a **unique deposit address per order**, which is simplest.

Step 3: Create an order and a deposit address

```
// server/orders.js
import express from "express";
import { randomUUID } from "node:crypto";
import { ChainForge } from "@chainforge/sdk";
import { deriveDepositAddress } from "./wallets.js"; // derives from KMS or
HD wallet

const client = new ChainForge({ apiKey: process.env.CHAINFORGE_API_KEY });
const TOKEN = "0x8464135c8F25Da09e49BC8782676a84730C318bC";
const router = express.Router();

router.post("/orders", async (req, res) => {
  const { bookId } = req.body;
  const book = await db.books.find(bookId);
  if (!book) return res.status(404).json({ error: "Book not found" });

  const orderId = randomUUID();
  const depositAddress = await deriveDepositAddress(orderId);

  // Price stored as integer token units: $12.50 -> 12_500_000 (6 decimals)
  const order = await db.orders.insert({
    id: orderId,
    bookId,
    status: "awaiting_payment",
    amountDue: book.priceUnits,
    depositAddress,
    expiresAt: new Date(Date.now() + 30 * 60 * 1000),
  });

  // Ask the platform to watch this address for token transfers
```

```
  await client.accounts.register({
    address: depositAddress,
    label: `order-${orderId}`,
    metadata: { orderId },
  });

  res.status(201).json({
    orderId,
    pay: {
      token: TOKEN,
      symbol: "ACMD",
      to: depositAddress,
      amount: order.amountDue.toString(),
      amountFormatted: "12.50",
      network: "ember",
      expiresAt: order.expiresAt,
    },
  });
});

export default router;
```

Step 4: Register a webhook for token transfers

```
curl -X POST https://api.chainforge.example/v1/webhooks \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://shop.acme.example/hooks/chainforge",
    "events": ["token.transfer"],
    "filters": { "tokens": ["0x8464135c8F25Da09e49BC8782676a84730C318bC"],
"finality": "finalized" }
  }'
```

Save the returned `secret` as `CHAINFORGE_WEBHOOK_SECRET`.

Step 5: Handle the webhook

Reuse the signature verification from section 8.3, then process the event idempotently:

```
async function handleTokenTransfer(event) {
  const t = event.data; // { token, from, to, value, transactionHash,
logIndex, blockNumber }

  const firstTime = await
db.processedEvents.insertIfAbsent(`${t.transactionHash}:${t.logIndex}`);
  if (!firstTime) return;

  const order = await db.orders.findByDepositAddress(t.to);
  if (!order) return; // unrelated transfer

  const paid = BigInt(t.value);
  const due = BigInt(order.amountDue);

  await db.payments.insert({ orderId: order.id, txHash: t.transactionHash,
units: paid.toString() });

  const totalPaid = await db.payments.sumUnits(order.id);

  if (order.status === "expired" || new Date() > order.expiresAt) {
    return db.orders.update(order.id, { status: "paid_late" }); // needs
manual review or refund
  }
  if (totalPaid < BigInt(due)) {
    return db.orders.update(order.id, { status: "underpaid" });
  }
  if (totalPaid >= due) {
    await db.orders.update(order.id, { status: "paid" });
    await deliverEbook(order); // e.g., email a signed download link
    if (totalPaid > due) await db.refunds.queue(order.id, totalPaid - due);
  }
}
```

Step 6: Test the flow

1. Create an order: `curl -X POST https://shop.acme.example/orders -d '{"bookId":"b_100"}' -H "Content-Type: application/json"`.
2. Copy the `pay.to` address and send the exact amount from the customer wallet:

   ```
   forge call 0x8464...18bC transfer <depositAddress> 12500000 --from
   customer
   ```

3. Watch the Console's **Webhook delivery log**. A `token.transfer` event should arrive about 4 seconds later.
4. Confirm the order becomes `paid` and the download is delivered.
5. Repeat with **too little** (expect `underpaid`), **too much** (expect a queued refund) and **after expiry** (expect `paid_late`).
6. Replay the same webhook from the Console and confirm that nothing is processed twice.

Hardening for production

Concern	Action
Wrong token	Match the contract address, not only the symbol. Anyone can deploy a token named "ACMD".
Reorgs and finality	Act only on `finalized` events.
Sweeping funds	Move balances from deposit addresses to a treasury through a controlled job, and keep gas funded.
Key security	Derive deposit addresses from KMS-held keys; no raw keys in code.
Reconciliation	Run an hourly job that lists token transfers and compares them with your payments table.
Accounting	Store amounts as integer token units and the token's `decimals` with each record.

10.2 Build a Simple NFT Minting App

Scenario. You organize a conference. Each attendee receives an NFT ticket after registering. Staff scan the ticket at the door to verify ownership.

You will learn: creating an NFT collection, preparing metadata, minting from a backend, letting a user view their ticket and verifying ownership at check-in.

Prerequisites: chapters 6 and 7; a funded Ember wallet for the organizer; Node.js 18+.

Step 1: Create the collection

```
forge token create --type cft721 --name "Acme Conference Tickets" --symbol
ACT \
  --max-supply 500 --network ember --from organizer
# Collection deployed at: 0x9A676e781A523b5d0C0e43731313A708CB607508
```

The `--max-supply 500` cap makes the supply limit publicly verifiable. A cap cannot be raised later.

Step 2: Prepare the metadata

Create one JSON file per ticket, for example `metadata/42.json`:

```
{
  "name": "Acme Conference 2026 - Ticket #42",
  "description": "Admits one person to Acme Conference 2026 on 3 December
2026, Bengaluru.",
  "image":
"ipfs://bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqabf3oclgtqy55fbzdi/ticket.pn
g",
  "attributes": [
    { "trait_type": "Tier", "value": "General" },
    { "trait_type": "Event", "value": "Acme Conference 2026" }
  ]
}
```

Upload the image and metadata folder to content-addressed storage and note the resulting folder address, such as `ipfs://bafybeih5kq.../`. Do **not** put the attendee's name, email or other personal data in public metadata. NFT data is permanent and public.

Step 3: Build the minting endpoint

```
// server/mint.js
import express from "express";
import { ChainForge, Wallet } from "@chainforge/sdk";
import { isAddress, toChecksumAddress } from "@chainforge/sdk";

const client = new ChainForge({ apiKey: process.env.CHAINFORGE_API_KEY });
const COLLECTION = "0x9A676e781A523b5d0C0e43731313A708CB607508";
const BASE_URI = "ipfs://bafybeih5kq.../";
const router = express.Router();
```

```
router.post("/register", async (req, res) => {
  const { email, walletAddress } = req.body;

  if (!isAddress(walletAddress)) {
    return res.status(400).json({ error: "Invalid wallet address" });
  }

  // Allow one ticket per registration, enforced in your own database
  const reg = await db.registrations.findByEmail(email);
  if (!reg || reg.ticketMinted) {
    return res.status(409).json({ error: "No eligible registration" });
  }

  const tokenId = await db.counters.next("ticket"); // 1, 2, 3 ...
  const signer = await getOrganizerSigner();        // KMS-backed in
production
  const tickets = client.contracts.at(COLLECTION, cft721Abi);

  const tx = await tickets.write.mint(
    [toChecksumAddress(walletAddress), BigInt(tokenId),
`${BASE_URI}${tokenId}.json`],
    { signer, idempotencyKey: `mint-${reg.id}` }
  );

  await db.registrations.update(reg.id, { ticketMinted: true, tokenId,
mintTx: tx.id });
  res.status(202).json({ tokenId, transactionId: tx.id, status: "minting" });
});

export default router;
```

Note the `idempotencyKey` derived from the registration ID. If the request is retried, the platform returns the first transaction instead of minting a second ticket.

Step 4: Show the user their ticket

```
router.get("/my-tickets", async (req, res) => {
  const wallet = req.query.address;
  const { data } = await client.accounts.listNfts(wallet, { collection:
COLLECTION });
  res.json(data.map(n => ({ tokenId: n.tokenId, name: n.name, image: n.image
})));
});
```

If the user does not yet have a wallet, offer a simple path: create a managed wallet for them with a custodial provider, or show instructions for installing a wallet application. Decide which fits your audience before you build.

Step 5: Verify at the door

At check-in, the attendee proves they control the wallet that owns the ticket. Ownership alone is not enough, since a screenshot of a ticket ID can be copied. A **signed challenge** proves control.

1. The door scanner shows a QR code containing a random challenge: `checkin:acme2026:8f21c9:exp=1795047000`.
2. The attendee's wallet signs the challenge (section 4.3) and displays the signature.
3. The scanner sends the challenge and signature to your server:

```
router.post("/check-in", async (req, res) => {
  const { challenge, signature, tokenId } = req.body;

  if (!(await db.challenges.consume(challenge))) {   // one-time use and not
expired
    return res.status(400).json({ valid: false, reason: "Bad or expired
challenge" });
  }

  const signer = verifyMessage(challenge, signature);
  const tickets = client.contracts.at(COLLECTION, cft721Abi);
  const owner = await tickets.read.ownerOf([BigInt(tokenId)], { block:
"finalized" });

  if (owner.toLowerCase() !== signer.toLowerCase()) {
    return res.status(403).json({ valid: false, reason: "Signer does not own
this ticket" });
```

```
  }
  if (await db.checkins.exists(tokenId)) {
    return res.status(409).json({ valid: false, reason: "Ticket already used"
});
  }

  await db.checkins.insert({ tokenId, at: new Date() });
  res.json({ valid: true });
});
```

Step 6: Test

1. Register with an Ember wallet address. Within about 4 seconds the explorer shows the ticket under the collection.
2. Query `/my-tickets` for that wallet and confirm the ticket is returned.
3. Run the check-in flow. Expect `valid: true`.
4. Replay the same challenge. Expect an error.
5. Transfer the ticket to a second wallet, and confirm that the first wallet can no longer check in.
6. Try to register twice with the same email. Expect `409`.

Design questions to settle before launch

- **Transferability.** Should tickets be freely transferable, or locked to the first owner? A non-transferable ("soulbound") option reduces scalping but also blocks legitimate gifting.
- **Refunds.** If someone is refunded, you must burn or invalidate their ticket. Plan the contract permissions for that.
- **Privacy.** Wallet addresses on a public chain can be linked to a person. Tell attendees what is recorded.

10.3 Listen to Events and Update a Database

Scenario. You run a loyalty program with the `LoyaltyPoints` contract from chapter 6. You want a fast, queryable database of every customer's balance and history so that your website does not query the chain on every page load.

You will learn: subscribing to contract events, storing them idempotently, resuming after downtime, backfilling history and verifying your database against on-chain truth.

Design principle. Treat the chain as the source of truth and the database as a **derived read model** that you can always rebuild from events.

Step 1: Create the schema

```
CREATE TABLE point_events (
  tx_hash       TEXT        NOT NULL,
  log_index     INTEGER     NOT NULL,
  block_number  BIGINT      NOT NULL,
  event_type    TEXT        NOT NULL,   -- 'issued' | 'transferred'
  from_address  TEXT,
  to_address    TEXT        NOT NULL,
  amount        NUMERIC(78,0) NOT NULL,
  occurred_at   TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (tx_hash, log_index)    -- guarantees de-duplication
);

CREATE TABLE point_balances (
  address     TEXT PRIMARY KEY,
  balance     NUMERIC(78,0) NOT NULL DEFAULT 0,
  updated_at  TIMESTAMPTZ NOT NULL
);

CREATE TABLE sync_state (
  name        TEXT PRIMARY KEY,
  last_block  BIGINT NOT NULL            -- last finalized block fully
processed
);
```

The primary key on `(tx_hash, log_index)` is what makes processing idempotent.

Step 2: Backfill history

Before going live, load all past events from the contract's deployment block.

```
import { ChainForge } from "@chainforge/sdk";
const client = new ChainForge({ apiKey: process.env.CHAINFORGE_API_KEY });
const CONTRACT = "0x5FbDB2315678afecb367f032d93F642f64180aa3";

async function backfill(fromBlock) {
  const iterator = client.contracts.listEvents(CONTRACT, {
    fromBlock,
    toBlock: "finalized",
    events: ["PointsIssued", "PointsTransferred"],
  });
  for await (const ev of iterator) {
    await applyEvent(ev);
  }
  const head = await client.blocks.getFinalized();
  await db.query("UPDATE sync_state SET last_block = $1 WHERE name =
'points'", [head.number]);
}
```

Step 3: Apply events atomically

```
async function applyEvent(ev) {
  await db.transaction(async (tx) => {
    const inserted = await tx.query(
      `INSERT INTO point_events (tx_hash, log_index, block_number,
event_type, from_address, to_address, amount, occurred_at)
       VALUES ($1,$2,$3,$4,$5,$6,$7,$8)
       ON CONFLICT (tx_hash, log_index) DO NOTHING
       RETURNING 1`,
      [ev.transactionHash, ev.logIndex, ev.blockNumber,
       ev.name === "PointsIssued" ? "issued" : "transferred",
       ev.args.from ?? null, ev.args.to, ev.args.amount, ev.timestamp]
    );
    if (inserted.rowCount === 0) return; // duplicate: nothing more to do

    if (ev.name === "PointsIssued") {
      await credit(tx, ev.args.to, ev.args.amount);
    } else {
      await debit(tx, ev.args.from, ev.args.amount);
      await credit(tx, ev.args.to, ev.args.amount);
    }
  });
}
```

```
async function credit(tx, address, amount) {
  await tx.query(
    `INSERT INTO point_balances (address, balance, updated_at) VALUES
($1,$2,now())
     ON CONFLICT (address) DO UPDATE SET balance = point_balances.balance +
$2, updated_at = now()`,
    [address.toLowerCase(), amount]
  );
}
async function debit(tx, address, amount) {
  await tx.query(
    `UPDATE point_balances SET balance = balance - $2, updated_at = now()
WHERE address = $1`,
    [address.toLowerCase(), amount]
  );
}
```

The event insert and the balance update share **one database transaction**. Either both happen or neither does.

Step 4: Go live with a subscription

```
async function startListening() {
  const { last_block } = await db.one("SELECT last_block FROM sync_state
WHERE name = 'points'");

  // Catch up on anything missed while the service was down
  await backfill(last_block + 1);

  const sub = client.subscribe({
    type: "contract.event",
    contract: CONTRACT,
    events: ["PointsIssued", "PointsTransferred"],
    finality: "finalized",
    fromBlock: last_block + 1,
  });

  sub.on("event", async (ev) => {
    await applyEvent(ev);
    await db.query("UPDATE sync_state SET last_block = GREATEST(last_block,
$1) WHERE name = 'points'", [ev.blockNumber]);
```

```
  });

  sub.on("error", (e) => logger.error({ err: e }, "subscription error"));
}
```

Step 5: Verify against the chain

A nightly job compares your balances with on-chain truth for a sample of addresses, or all of them if the set is small:

```
async function reconcile() {
  const rows = await db.many("SELECT address, balance FROM point_balances");
  const points = client.contracts.at(CONTRACT, abi);
  const mismatches = [];
  for (const row of rows) {
    const onChain = await points.read.balanceOf([row.address], { block:
"finalized" });
    if (onChain.toString() !== row.balance.toString()) {
      mismatches.push({ address: row.address, db: row.balance, chain:
onChain.toString() });
    }
  }
  if (mismatches.length) alert("Balance drift detected", mismatches);
}
```

If drift appears, you can **rebuild** the read model: truncate the tables, reset `last_block` to the contract's deployment block, and run the backfill again.

Step 6: Test the pipeline

1. Start the listener and the devnet or Ember.
2. Issue 100 points to an address: `forge call <contract> issue <address> 100 --from alice`.
3. Within seconds `point_balances` shows 100 for that address.
4. Transfer 40 points onward and confirm both balances update.
5. **Stop the listener**, issue more points, then restart. Confirm the backfill brings the database up to date with no gaps or duplicates.
6. Run `reconcile()` and confirm there are no mismatches.

Pitfalls

Pitfall	Consequence	Prevention
Using `included` instead of `finalized` events	Rare corrections after the fact	Use `finalized` for financial data
Storing amounts as floats	Rounding errors	`NUMERIC` in SQL, BigInt in code
Updating `last_block` before applying the event	Lost events after a crash	Update it after, in the same transaction if possible
No unique key on events	Duplicates after reconnects	Primary key on (`tx_hash`, `log_index`)
Skipping reconciliation	Silent drift	Scheduled verification with alerts

---

11. Error Handling and Troubleshooting

Things will go wrong: networks time out, users mistype addresses, contracts revert. This chapter helps you recognize each kind of failure, find the cause quickly and fix it.

11.1 Error Format and Common Codes

Error response format

Every error from the REST API uses the same JSON structure:

```
{
  "error": {
    "code": "insufficient_funds",
    "message": "The sender balance is too low to cover value plus maximum
fee.",
    "details": {
      "required": "5063000000000000000",
      "available": "4200000000000000000"
    },
    "requestId": "req_5GT7M1K",
    "docUrl": "https://docs.chainforge.example/errors#insufficient_funds"
  }
}
```

Field	Meaning
`code`	Stable, machine-readable identifier. **Branch on this**, never on `message`.
`message`	Human-readable explanation. May change over time.
`details`	Optional structured data about the failure
`requestId`	Unique ID for this request. **Always include it when contacting support.**
`docUrl`	Link to the explanation of this error

HTTP status codes

Status	Meaning	Retry?
`200` / `201` / `202`	Success, created, or accepted for processing	n/a
`204`	Success with no content	n/a
`400`	Malformed request (invalid JSON, wrong type)	No. Fix the request.

Status	Meaning	Retry?
`401`	Missing, invalid or expired credentials	No. Fix credentials.
`403`	Valid credentials but not permitted	No. Check scopes and IP rules.
`404`	Resource does not exist	No, unless eventual consistency may apply (see below)
`409`	Conflict (idempotency mismatch, duplicate)	Depends on `code`
`422`	Well-formed but fails validation or the call would revert	No. Fix the input.
`429`	Rate limit exceeded	Yes, after `Retry-After`
`500` / `502` / `503` / `504`	Platform or upstream problem	Yes, with backoff

Error code catalog (selected)

Code	HTTP	Likely cause	Remedy
`invalid_api_key`	401	Typo, revoked key or wrong environment	Re-copy the key; check the `cfk_test_` or `cfk_live_` prefix
`token_expired`	401	OAuth token older than one hour	Request a new token
`insufficient_scope`	403	Key or token lacks the required scope	Create a key with the correct permissions
`ip_not_allowed`	403	Request came from an address outside the allow-list	Update the key's IP rules
`rate_limited`	429	Too many requests	Back off and retry
`invalid_address`	400	Not a valid 42-character address or failed checksum	Validate input before sending

Code	HTTP	Likely cause	Remedy
`invalid_amount`	400	Negative, non-integer or too large	Use integer sparks as a string
`wrong_network`	422	Transaction chain ID does not match the key's network	Sign with the correct chain ID
`nonce_too_low`	422	Nonce already used	Fetch the current nonce
`nonce_gap`	422	A lower nonce is missing	Send the missing transaction or cancel
`insufficient_funds`	422	Balance below value plus maximum fee	Fund the account or lower the fee ceiling
`fee_too_low`	422	Fee ceiling below the current base fee	Re-estimate and resubmit
`gas_limit_too_low`	422	Limit cannot cover even the base cost	Increase `gasLimit`
`execution_reverted`	422	Contract rejected the call	Read `details.revertReason`
`idempotency_conflict`	409	Same key reused with a different payload	Use a new key for a new operation
`already_known`	409	This exact transaction is already in the mempool	Track the existing transaction
`contract_not_registered`	404	No ABI registered for this address	Register the contract
`webhook_unreachable`	422	Endpoint failed the creation-time check	Confirm HTTPS, public reachability and a `2xx` response
`service_unavailable`	503	Temporary platform problem	Retry with backoff; see the status page

Eventual consistency

A transaction you just submitted may briefly return `404` from a different API server before it propagates. If you receive `404` for an ID you just created, retry a few times over two or three seconds before treating it as an error.

11.2 Debugging Failed Transactions

Work through these steps in order. Most problems are found by step 4.

Step 1: Identify the stage

Where did it fail?	Symptom	Go to
Before submission	The API rejected the request (`4xx`)	Step 2
In the mempool	Status stays `pending` or becomes `dropped`	Step 3
In execution	Status `failed` with a revert reason	Step 4
After finality	Transaction succeeded but the outcome is wrong	Step 5

Step 2: Rejected before submission

1. Read `error.code` and `error.details`.
2. Confirm the **network**: does the key prefix, the `chainId` in your signed transaction and the faucet/explorer you are using all refer to the same network?
3. Confirm the **nonce**: `GET /v1/accounts/{address}` returns the confirmed nonce; compare with the one you used.
4. Confirm the **balance** covers `value + gasLimit x maxFeePerGas`.
5. Confirm the **amount unit**: a value of `5` means five sparks, not five FORGE.

Step 3: Stuck in the mempool

See section 5.5: check the fee against the base fee, check for nonce gaps, then speed up or cancel.

Step 4: Reverted in execution

1. Fetch the transaction and read `revertReason`.

2. If it is empty, open the transaction in the **block explorer** and look at the trace to find the exact failing instruction.
3. **Simulate** the same call at the block before it failed:

```
const sim = await contract.simulate.issue([addr, 100n], { from: sender,
block: failedBlock - 1 });
console.log(sim.revertReason);
```

4. Check the likely causes in order:
   - Is the **caller** the account you expect? (Access control reverts are the most common.)
   - Do the **arguments** satisfy the contract's `require` checks?
   - Does the contract have **enough balance or allowance**?
   - Is the contract **paused**, or did a state change since you tested?
   - Did it run **out of gas**? Compare `gasUsed` with `gasLimit`.
5. Reproduce on the **local devnet** by forking Ember at the failing block (`forge devnet start --fork ember --fork-block <n>`), then step through with logging.

Step 5: Succeeded, but wrong result

- Check the **event logs**. Did the expected events fire with the expected arguments?
- Check **decimals and units** (a 6-decimal token amount sent as if it had 18 decimals is a classic error).
- Check that your application read state at the right finality level. Reading `latest` right after a write can miss the update. Reading `finalized` waits a few seconds longer.
- Check whether the transaction was **replaced** by a different one with the same nonce.

Troubleshooting by symptom

Symptom	Probable causes	What to try
`401` on every call	Wrong or revoked key, whitespace in the header, bearer prefix missing	Print the first 10 characters of the key, confirm `Authorization: Bearer <key>`
Faucet says "limit reached"	24-hour limit per address	Use another address or the local devnet

Symptom	Probable causes	What to try
Balance is not updated	Reading `latest` too soon, or the transaction is still pending	Wait for `finalized`; confirm in the explorer
`nonce_too_low` after a restart	Your local nonce counter reset	Re-fetch the nonce from the API (`block=pending`)
Webhook never arrives	Endpoint not public, wrong URL, TLS error, filter too narrow	Check the delivery log and use **Send test event**
Webhook signature fails	Body parsed before verification, wrong secret, or clock skew	Verify against the raw bytes and sync server time
Duplicate processing	Missing idempotent handling	Store event IDs (section 8.4)
WebSocket drops every few minutes	Idle timeout through a proxy	Enable the SDK's heartbeat; allow long-lived connections
Contract call returns zero or empty	Wrong address, wrong network or contract not yet deployed	Check the explorer for code at that address on this network
`contract_not_registered`	ABI never registered	Run `forge contract register` or pass the ABI to the SDK

Information to collect before contacting support

1. The `requestId` from the failing response.
2. The transaction hash or ID, and the network.
3. UTC time of the failure.
4. Your SDK name and version (`npm ls @chainforge/sdk` or `pip show chainforge-sdk`).
5. What you expected and what happened.
6. **Never** send private keys, recovery phrases or full API keys.

11.3 FAQs

Why does my transaction show "pending" for a long time? Usually the fee is below the current base fee, or an earlier nonce is missing. See section 5.5.

I sent funds to the wrong address. Can they be recovered? No. ChainForge transactions are final and the platform cannot reverse them. Only the owner of the receiving address can return funds, so validate addresses before sending and send a small test amount first for large transfers.

Do I pay gas for failed transactions? Yes, for the gas actually consumed up to the point of failure. Simulate calls first to avoid this. Transactions that are `dropped` cost nothing.

What is the difference between "included" and "finalized"? Included means the transaction is in a block. Finalized means that block can never be reversed. Use finalized for anything with financial consequences.

Can I use Ethereum libraries such as ethers.js or web3.py? Yes. ChainForge is EVM-compatible and exposes a standard JSON-RPC endpoint. Use chain ID 7700, 7701 or 7702, as appropriate. Platform conveniences such as webhooks, gas suggestions and idempotent submission require the REST API or the ChainForge SDKs.

Can I use the same API key for testnet and mainnet? No. Keys are bound to one environment, which is intentional.

How many confirmations should I wait for? None in the traditional sense. Wait for the `finalized` status, which takes about 4 seconds.

How do I get test tokens when the faucet limit is reached? Use the local devnet, which gives unlimited free tokens, or ask in the developer community channel for a one-time top-up.

Are amounts strings or numbers in the API? Strings containing integers in the smallest unit, to avoid precision loss. Convert with the SDK helpers.

Can I delete or edit data on the chain? No. Data on a blockchain is permanent. Never write personal data on-chain; store it off-chain and keep only a hash or reference on-chain.

How do I change a deployed contract? You cannot change deployed code, but you can design for upgrades with proxies or migration (section 6.6).

Where can I check whether ChainForge is having an incident? The status page at `https://status.chainforge.example` shows current and historical incidents for the API, the networks and the faucet. You can subscribe to email or webhook notifications.

---

12. Going to Production

Moving from Ember to Forge Mainnet means real value is at stake. This chapter gives you a checklist and practical guidance on monitoring, performance and support.

12.1 Mainnet Readiness Checklist

Complete every section before launch. Record the date and the owner of each item.

Product and legal

- [ ] The compliance and legal implications of handling digital assets in your target markets have been reviewed by qualified advisors.
- [ ] Terms of service, privacy policy and risk disclosures are published.
- [ ] User-facing text explains that blockchain transactions are final and irreversible.
- [ ] A data-protection review confirms that **no personal data is written on-chain**.

Environment and configuration

- [ ] A separate **live** project and `cfk_live_` keys exist, stored in a secrets manager.
- [ ] Configuration verifies the **chain ID (7700)** at startup and refuses to run against the wrong network.
- [ ] Mainnet contract addresses are read from configuration, not hard-coded from testnet.
- [ ] Test, staging and production environments are fully separated.

Security

- [ ] Every item in the security checklist (section 4.5) is complete.
- [ ] Production signing keys live in a KMS, HSM or hardware wallet. No key is on a developer machine.
- [ ] Multi-signature approval protects treasury transfers and contract administration.
- [ ] Contracts holding meaningful value have passed an **independent audit**, and all findings are resolved or formally accepted.

- [ ] An **incident response runbook** covers key compromise, contract pause, webhook secret rotation and user communication.
- [ ] A bug-bounty or responsible-disclosure channel is available.

Reliability

- [ ] End-to-end tests pass on Ember using the **same code and configuration shape** as production.
- [ ] Idempotency keys are used for every state-changing request.
- [ ] Webhook handlers are idempotent, verify signatures and acknowledge quickly.
- [ ] A reconciliation job compares your records with the chain on a schedule.
- [ ] Retry logic handles `429` and `5xx` with exponential backoff and jitter.
- [ ] Timeouts and circuit breakers prevent a slow dependency from taking your service down.

Operations

- [ ] Dashboards and alerts are live (section 12.2).
- [ ] Runbooks exist for common problems: stuck transactions, webhook failures, low wallet balance.
- [ ] On-call responsibilities and escalation paths are defined.
- [ ] The hot-wallet float is funded with enough FORGE for expected gas, and alerts fire when it runs low.
- [ ] A **small-value dress rehearsal** on mainnet succeeded: real transactions with minimal amounts, verified end to end.

Launch plan

- [ ] Staged rollout (internal users, then a small percentage, then everyone).
- [ ] A rollback or kill-switch exists for each new feature.
- [ ] Support teams are trained on the most likely user questions.
- [ ] The plan names who decides to pause the launch and under what conditions.

12.2 Monitoring and Logging

What to monitor

Area	Metric	Alert when
API health	Error rate by status code, p95 latency	`5xx` above 1 percent for 5 minutes; latency twice its baseline
Rate limits	Count of `429` responses	Any sustained increase
Transactions	Time from submission to finalized; share that fail	Median above 15 seconds; failure rate above 2 percent
Pending queue	Number of transactions pending longer than 2 minutes	Greater than zero for 10 minutes
Wallets	Hot-wallet balance in FORGE	Below a defined minimum
Webhooks	Delivery success rate; retry queue depth; processing lag	Success below 99 percent; lag above 1 minute
Event pipeline	Last processed block versus finalized head	More than 30 blocks behind
Reconciliation	Mismatches between database and chain	Any mismatch
Contracts	Unexpected admin function calls, large transfers, paused state	Any admin call; transfers above a threshold
Security	API calls from unfamiliar IPs, key usage spikes	Anomaly against baseline

Logging guidelines

Do log

- `requestId`, transaction ID and hash, status transitions with timestamps
- Network and chain ID
- Business identifiers (order ID, customer ID) next to technical ones
- Webhook event IDs and processing outcome
- Retries and the reasons for them

Do not log

- Private keys, recovery phrases, keystore passwords
- Full API keys or webhook secrets (log only the last four characters)
- Personal data you do not need

Use **structured (JSON) logs** with consistent field names so you can search and aggregate them:

```
{
  "level": "info",
  "ts": "2026-10-18T10:21:07Z",
  "msg": "transaction finalized",
  "orderId": "ord_8841",
  "txId": "txn_01JC9M6A1B",
  "txHash": "0x4e1d...a9c3",
  "network": "mainnet",
  "submitToFinalMs": 4120
}
```

Platform tools

- **Console → Usage:** request volume, errors and latency by key and endpoint.
- **Console → Webhooks → Delivery logs:** every attempt with request, response and timing.
- **Status page:** subscribe so incidents reach your on-call channel automatically.
- **Usage API:** `GET /v1/usage` returns metrics you can import into your own monitoring.

12.3 Performance and Scaling Tips

Reduce the number of calls

1. **Subscribe instead of poll** (chapter 8).
2. **Batch reads.** Use the multi-address balances endpoint and multicall-style contract reads.
3. **Cache immutable data.** ABIs, chain ID, token decimals, names and symbols almost never change. Cache them for hours or days.
4. **Cache finalized data forever.** A finalized block or transaction never changes.
5. **Use cursors with large page sizes** (up to 100) rather than many small requests.

Send transactions efficiently

Technique	Benefit
Several sender wallets in parallel	Avoids nonce bottlenecks; scales throughput linearly
Local nonce management	Removes a round trip per transaction
Batch contract calls (for example `batchMint`)	One transaction instead of hundreds; lower total gas
`slow` fee tier for non-urgent work	Cheaper batch operations
Queue and rate-limit submissions	Keeps you inside quotas and gives smooth load

Keep gas costs down

- Store as little as possible on-chain; use events and off-chain storage for bulk data.
- Use compact data types and pack related values into one storage slot.
- Avoid unbounded loops in contracts. Gas grows with the size of data, and a loop that works with 10 items may fail with 10,000.
- Prefer `calldata` to `memory` for external function arrays.
- Measure with the test tools (`forge test --gas-report`) and watch for regressions.

Design for the latency you have

- Finalization takes about 4 seconds. Show a **pending state** in the interface and update it from events, rather than blocking the user.
- Acknowledge webhooks immediately and process in the background.
- Make interfaces tolerant of `included` first and `finalized` shortly after.

Capacity planning

1. Estimate peak requests per second, including retries, and compare against your plan limit with headroom of at least 30 percent.
2. Load-test on the **local devnet** for your own code and on **Ember** for end-to-end behavior, but contact support before large-scale tests on shared networks.
3. Move up a plan, or request custom limits, before a launch or campaign rather than during it.

12.4 Support and SLAs

Support channels

Channel	Best for	Availability
Documentation and API Reference	Self-service answers	Always
Community forum	Questions and sharing patterns	Community-driven
Console support tickets	Account, billing and technical problems	By plan (below)
Status page	Incident information	Real time
Security contact (`security@chainforge.example`)	Vulnerabilities and key-compromise reports	24x7

Plans and service levels

	Free	Growth	Enterprise
Networks	Ember, devnet	Ember, devnet, mainnet	All, plus dedicated capacity
API availability target	Best effort	99.9% monthly	99.99% monthly
Support channel	Community	Email tickets	Tickets plus dedicated engineer
First response target	None	1 business day (critical: 4 hours)	1 hour for critical issues, 24x7
Rate limit	10 req/s	50 req/s	Custom
Service credits for missed SLA	No	Yes	Yes

Severity levels

Severity	Definition	Example
Critical	Production down or funds at risk; no workaround	All transactions failing; suspected key compromise
High	Major feature degraded; workaround is costly	Webhooks delayed by hours
Normal	Limited impact, workaround exists	Incorrect value in one response field
Low	Question or enhancement request	Documentation clarification

When you file a ticket, state the severity, include the information listed in section 11.2 and say whether production is affected.

Versioning and deprecation policy

- The API is versioned in the URL (`/v1`). Within a version, changes are **backwards compatible**: new fields, endpoints and optional parameters. Write clients that ignore fields they do not recognize.
- **Breaking changes** ship only in a new major version. The previous version is supported for at least **12 months** after the successor is released.
- Deprecations are announced in the changelog, by email to project administrators and with `Deprecation` and `Sunset` headers on affected responses.
- SDKs follow semantic versioning. Pin to a minor version in production and test upgrades on Ember before adopting them.

Network upgrades

Occasionally the protocol itself is upgraded (a "hard fork"). These are announced at least four weeks ahead for mainnet and two weeks ahead for Ember, with a published block height or time. Review the release notes for changes that affect gas costs or contract behavior, test on Ember first and confirm that your SDK version supports the new rules.

Appendices

Appendix A: Glossary

Term	Definition
ABI (Application Binary Interface)	A JSON description of a contract's functions, events and errors, needed to encode calls and decode results
Address	A 42-character identifier of an account, beginning with `0x`
Allowance	The amount of a token one address has approved another to spend on its behalf
API key	A secret credential that identifies your project to the ChainForge API
Base fee	A protocol-set minimum price per unit of gas that adjusts with demand and is burned
Block	A batch of transactions linked to the previous block by a hash
Burn	To permanently destroy tokens or fees, removing them from supply
BFT (Byzantine fault tolerance)	The ability of a system to reach agreement even when some participants fail or act maliciously
Calldata	The input data included with a transaction or contract call
Chain ID	A number that identifies a network and prevents transactions from being replayed on another network
Confirmation	An additional block built on top of the block containing a transaction
Consensus	The process by which validators agree on the next block

Term	Definition
Contract account	An account controlled by code rather than by a private key
Devnet	A local, single-node development network
EOA (Externally owned account)	An account controlled by a private key
EVM	Ethereum Virtual Machine; the execution environment that runs contract bytecode
Event / log	A record emitted by a contract during a transaction, readable by outside systems
Faucet	A service that gives free test tokens on a test network
Finality	The point at which a transaction can no longer be reversed
FORGE	The native token of ChainForge
Fungible	Interchangeable with other units of the same kind
Gas	A unit measuring computational work; the basis of transaction fees
Gas limit	The maximum gas a transaction may consume
Hot wallet / cold wallet	A wallet whose key is available to a running system, versus one kept offline
Idempotency key	A unique value that makes repeated requests produce a single effect
Keystore	An encrypted file that holds a private key
KMS / HSM	Key management service or hardware security module that holds keys and signs on request
Mainnet	The production network with real economic value
Mempool	The pool of valid transactions waiting to be included in a block
Mint	To create new tokens

Term	Definition
Multi-signature (multisig)	A wallet that requires several keys to approve an action
Nonce	A per-account transaction counter used for ordering and replay protection
NFT	Non-fungible token; a unique token with one owner
Proof of stake	A consensus approach in which validators lock up tokens as a guarantee of honest behavior
Proxy contract	A contract that forwards calls to a replaceable implementation, enabling upgrades
Replay attack	Reusing a valid signed message or transaction where it was not intended
Revert	The undoing of all changes made by a failed contract call
Slot	The time window (about 2 seconds) in which a validator may propose a block
Spark	The smallest unit of FORGE; 1 FORGE = 10^18 sparks
Testnet (Ember)	A public network for testing with valueless tokens
Time-lock	A delay enforced between announcing and executing a sensitive action
Token	A digital asset tracked by a smart contract
Validator	A node that stakes FORGE and participates in producing and approving blocks
Webhook	An HTTPS request sent by the platform to your server when an event occurs

Appendix B: Supported Networks and Chain IDs

Network	Chain ID	Native token	Block time	Finality	REST base URL	JSON-RPC	Explorer
Forge Mainnet	7700	FORGE	~2 s	~4 s	`https://api.chainforge.example/v1` (with `cfk_live_` key)	`https://rpc.chainforge.example`	`https://explorer.chainforge.example`
Ember Testnet	7701	tFORGE	~2 s	~4 s	`https://api.chainforge.example/v1` (with `cfk_test_` key)	`https://rpc.ember.chainforge.example`	`https://explorer.ember.chainforge.example`
Local Devnet	7702	dFORGE	Configurable (default instant)	Instant	`http://127.0.0.1:8545`	`http://127.0.0.1:8545`	Built-in local viewer

Adding ChainForge to a wallet application

Field	Mainnet	Ember
Network name	ChainForge Mainnet	ChainForge Ember Testnet
RPC URL	`https://rpc.chainforge.example`	`https://rpc.ember.chainforge.example`
Chain ID	7700	7701
Currency symbol	FORGE	tFORGE
Explorer URL	`https://explorer.chainforge.example`	`https://explorer.ember.chainforge.example`

Network parameters

Parameter	Value
Smallest unit	1 spark = 10^-18 FORGE
Minimum gas for a transfer	21,000
Block gas limit	30,000,000
Transaction expiry in the mempool	30 minutes
Maximum transaction size	128 KB
Address format	20 bytes, `0x` plus 40 hex characters, optional mixed-case checksum

Appendix C: Release Notes and Changelog

ChainForge Developer Platform

v1.4.0 (October 2026)

- Added `forge devnet --fork` with `--fork-block` to reproduce failures at a specific block.
- Added `transactions.speedUp` and `transactions.cancel` helpers to both SDKs.
- Webhook secrets can now be rotated with a 24-hour overlap.
- Improved revert-reason decoding for custom Solidity errors.

v1.3.0 (September 2026)

- Introduced `GET /v1/accounts/{address}/nfts` with collection filters.
- Added the `finality` option to WebSocket subscriptions.
- The Python SDK gained an asyncio client (`AsyncChainForge`).
- Raised the Free-tier limit from 5 to 10 requests per second.

v1.2.0 (July 2026)

- Added webhook delivery logs and manual redelivery.
- Added cursor-based pagination to all list endpoints.
- Introduced OAuth 2.0 client-credentials authentication and fine-grained scopes.

v1.1.0 (May 2026)

- Added token templates (`forge token create`) for CFT-20 and CFT-721.
- Added the `Idempotency-Key` header to all state-changing endpoints.

v1.0.0 (March 2026)

- General availability of Forge Mainnet, Ember testnet, the REST API and the JavaScript and Python SDKs.

> **Note:** This changelog is fictional and exists to demonstrate the format. A real changelog should list each change under headings such as *Added*, *Changed*, *Deprecated*, *Removed*, *Fixed* and *Security*, and flag breaking changes prominently.

Appendix D: Links to the API Reference Guide

The companion **ChainForge API Reference Guide** documents every endpoint in detail: parameters, request and response schemas, status codes and examples, generated from the OpenAPI 3.0 specification. Use this table to move from a task in this guide to its reference entries.

Task in this guide	Section	Endpoint(s) in the API Reference
Check API status and network	2.3	`GET /status`
Register an account	5.1	`POST /accounts`, `GET /accounts/{address}`
Check balances	5.2	`GET /accounts/{address}/balance`, `GET /accounts/balances`
Prepare and submit a transaction	5.3	`POST /transactions/prepare`, `POST /transactions`
Track a transaction	5.3	`GET /transactions/{id}`, `GET /accounts/{address}/transactions`
Estimate gas	5.4	`POST /gas/estimate`
Speed up or cancel	5.5	`POST /transactions/{id}/speed-up`, `POST /transactions/{id}/cancel`
Deploy and register contracts	6.3	`POST /contracts`, `POST /contracts/{address}/register`
Call and query contracts	6.4	`POST /contracts/{address}/call`, `POST /contracts/{address}/simulate`
Query contract events	6.5	`GET /contracts/{address}/events`
Create and inspect tokens	7.1	`POST /tokens`, `GET /tokens/{address}/balances/{holder}`
List NFTs	7.2	`GET /accounts/{address}/nfts`
Manage webhooks	8.2	`POST /webhooks`, `GET /webhooks`, `PATCH /webhooks/{id}`, `DELETE /webhooks/{id}`
Rotate a webhook secret	8.3	`POST /webhooks/{id}/rotate-secret`

Task in this guide	Section	Endpoint(s) in the API Reference
Redeliver a webhook event	8.4	`POST /webhooks/{id}/deliveries/{eventId}/redeliver`
Request an OAuth token	4.1	`POST /oauth/token`
Read usage metrics	12.2	`GET /usage`

The error codes in section 11.1 and the object schemas (Transaction, Account, Contract, Token, Webhook, Event, Error) are defined in the API Reference's *Models* section.

End of the ChainForge Developer Guide.